

C++ Deep Learning Engine

Build Log & Technical Report

From-Scratch Transformer with Curriculum Learning

Stack: C++17 · Pure STL · POSIX mmap · OpenMP · GTest · Python Preprocessor

Key Features:

- Reverse-mode autograd over a dynamic Directed Acyclic Graph (DAG)
- Full GPT-style Transformer with Fused Causal Self-Attention
- Curriculum Learning: Synthetic Math → TinyStories → Wikipedia
- OS-level `mmap` DataLoader with $O(1)$ RAM footprint
- No Eigen, no LibTorch, no Boost - zero ML dependencies

Development Phase: Phase 1 - Math Engine & Autograd Core

Micro-Steps Documented: 1.1 (Tensor) & 1.2 (Node)

August 12, 2026

Contents

1	Project Architecture Overview	5
1.1	Mission Statement	5
1.2	15-Step Development Roadmap	5
1.3	Repository Structure (Engine Sub-tree)	6
2	Step 1.1 - Tensor Foundation	7
2.1	Design Decisions	7
2.2	Row-Major Stride Formula	7
2.3	File: <code>engine/tensor.hpp</code>	8
2.4	File: <code>engine/tensor.cpp</code>	10
3	Step 1.2 - Autograd Node & Graph	12
3.1	Ownership Model & Memory Safety	12
3.1.1	Why <code>weak_ptr</code> in children?	13
3.2	Gradient Accumulation	13
3.3	File: <code>engine/node.hpp</code>	14
3.4	File: <code>engine/node.cpp</code>	17
4	Key Design Patterns (Interview Reference)	19
4.1	Smart Pointer Ownership Pattern	19
4.2	Gradient Accumulation vs. Assignment	19
4.3	Memory Layout - Cache Locality Impact	19
5	Build & Verification Instructions	19
5.1	Compile Steps 1.1 + 1.2 (Standalone Smoke Test)	19
5.2	Minimal Smoke Test (<code>smoke_test.cpp</code>)	20
6	What Comes Next	21
7	Step 1.3 - Core Differentiable Tensor Operations	22
7.1	The Autograd Contract	22
7.2	Critical Design: Cycle-Safe Backward Lambda Captures	22
7.2.1	The Cycle Problem	22
7.2.2	The Fix: <code>weak_ptr</code> Self-Capture	22
7.3	High-Performance Tiled MatMul	23
7.3.1	Tiling Algorithm (<code>mm_nn</code>)	23
7.4	Backward Derivations for All 7 Ops	24
7.5	File: <code>engine/ops.hpp</code>	24
7.6	File: <code>engine/ops.cpp</code> - Selected Implementations	25
8	Step 1.4 - Topological Backward Pass	27
8.1	Algorithm: DFS Post-Order Topological Sort	27
8.1.1	Why Post-Order?	27
8.1.2	Post-Order DFS Trace	27
8.2	Visited Set: <code>std::unordered_set<const Node*></code>	28

8.3	Handling <code>weak_ptr</code> Children	28
8.4	File: <code>engine/autograd.hpp</code>	29
8.5	File: <code>engine/autograd.cpp</code>	30
8.6	Correctness: The Gradient Seeding Invariant	31
8.7	End-to-End Smoke Test	31
9	Step 2.1 - Module Base, Linear, and LayerNorm	32
9.1	Module Base Class (<code>nn/module.hpp</code>)	32
9.2	Linear Layer	33
9.2.1	Forward Pass	33
9.2.2	Why a Custom <code>bias_add</code> Op?	33
9.2.3	Xavier/Glorot Uniform Initialisation	34
9.3	Layer Normalization	34
9.3.1	Forward Formula	34
9.3.2	Backward Derivation (Interview-Critical)	34
10	Step 2.2 - Embedding, Activation Functions, and Softmax	36
10.1	Embedding Layer	36
10.1.1	Forward: Gather	36
10.1.2	Backward: Scatter-Add	36
10.2	Activation Functions	37
10.3	Softmax (Numerically Stable)	37
10.3.1	Forward: Max-Subtraction Trick	37
10.3.2	Backward: Jacobian-Vector Product	37
11	Step 2.3 - Fused Causal Self-Attention	38
11.1	The 4-D Tensor Problem and Its Solution	38
11.2	File-Local Custom Ops	39
11.2.1	Head Permutation Index Mapping	39
11.3	Forward Algorithm (12 Steps)	39
11.4	Scaled Causal Softmax - Fused Op	40
11.4.1	Forward	40
11.4.2	Backward - Scale and Mask Chain Rule	40
12	Step 2.4 - FFN, TransformerBlock, and Full Transformer	41
12.1	FeedForward	41
12.2	Pre-LayerNorm Architecture	41
12.2.1	Forward pass of TransformerBlock	42
12.2.2	Parameter count per block	42
12.3	Full Transformer Language Model	42
12.3.1	Architecture	42
12.3.2	Forward pass (6 steps)	43
12.3.3	Total parameter NodePtrs	43
13	Step 3.1 - Python Preprocessor (<code>scripts/preprocess.py</code>)	44
13.1	Binary File Format	44
13.2	Character-Level Tokeniser	45
13.3	Dataset Modes	45
13.3.1	Math Mode - Synthetic Addition Equations	45

13.3.2	Stories Mode - roneneldan/TinyStories	45
13.3.3	Wiki Mode - Simple English Wikipedia	45
13.4	Caching and Safety	45
13.5	CLI Summary	46
14	Steps 3.2 & 3.3 - POSIX mmap DataLoader and Curriculum Scheduler	47
14.1	Step 3.2 - POSIX mmap DataLoader	47
14.1.1	Why mmap instead of read()	47
14.1.2	open_and_mmap - step by step	47
14.1.3	Header validation	48
14.1.4	Batch sampling - causal language modelling	48
14.1.5	switch_dataset - hot-swap	48
14.2	Step 3.3 - Curriculum Scheduler	48
14.2.1	Phase mapping	48
14.2.2	Training loop integration	49
15	Step 4.1 - Cross-Entropy Loss & AdamW Optimizer	50
15.1	Cross-Entropy Loss	50
15.1.1	Log-sum-exp trick - derivation	50
15.1.2	Single-pass forward	51
15.1.3	Analytically fused backward	51
15.2	AdamW Optimizer	51
15.2.1	AdamW vs Adam + L2	51
15.2.2	Update rule (one step)	52
15.2.3	Bias correction - why the first steps are stable	52
16	Step 4.2 - Main Curriculum Training Loop (main.cpp)	52
16.1	CLI Interface	53
16.2	Six-Step Training Inner Loop	53
16.3	Curriculum Phase Transition Logic	54
16.4	Checkpointing Format	54
16.5	NaN Guard	54
17	Step 5.1 - GTest Unit Tests: Math Engine & NN Modules	55
17.1	Test Files Overview	55
17.2	Numerical Gradient Checker	55
17.3	Causal Mask Verification Strategy	56
18	Step 5.2 - GTest Tests: DataLoader & Curriculum + Benchmarks	57
18.1	Test Files	57
18.1.1	test_dataloader.cpp - 12 tests	57
18.1.2	test_curriculum.cpp - 21 tests	58
18.2	Benchmark Files	58
18.2.1	bench_matmul.cpp - GFLOPS comparison	58
18.2.2	bench_attention.cpp - tokens/sec	58
18.2.3	bench_dataloader.cpp - GB/s	58
19	Step 5.3 - CMake Build System & REST Inference Server	60

19.1	CMakeLists.txt Design	60
19.2	Raw POSIX HTTP/1.1 Server	60
19.2.1	Socket lifecycle	60
19.2.2	HTTP/1.1 request framing	61
19.3	/predict Handler - Data Flow	61
19.3.1	Why argmax at position $T - 1$?	61
19.3.2	Naive JSON extraction	61
20	Bug Fixes & Stabilization (Post-Phase 1)	62
20.1	DataLoader FileHeader Original Layout Restoration	62
20.2	DataLoader Error Handling Use-After-Free (AddressSanitizer)	63
20.3	FeedForward Phantom GELU Member Compilation Error	63
20.4	CausalSelfAttention Zero-Gradient Mathematical Property	63
20.5	FeedForward Constructor Initialization Order	63
21	Interview Discussion: Architectural Trade-offs	64
21.1	Why "Zero Dependencies"?	64
21.2	Performance Optimizations: How AVX/BLAS Libraries Accelerate Com- pute	64
21.3	Future Scalability: CPU vs GPU Compute	65
21.4	1-Hour Curriculum Test Execution Log (1200 Steps)	65
22	Phase 6 - Cloud Deployment & Large-Scale Training	67
22.1	Cloud Hardware Selection	67
22.2	Deployment Pipeline	67
22.3	The "Golden Ratio" Training Command	67
	Project Complete	68

1 Project Architecture Overview

1.1 Mission Statement

This project implements a production-grade Deep Learning Engine entirely from scratch in C++17 - no Eigen, no LibTorch, no Boost. Every component, from the N-dimensional Tensor container to the multi-head attention mechanism, is built using only the C++ Standard Template Library.

1.2 15-Step Development Roadmap

The full project is divided into 15 granular micro-steps. Each step produces at most 2-3 files and is independently compilable and testable. This document records the implementation of Steps 1.1 and 1.2.

Step	Focus	Key Deliverables
1.1	Tensor Foundation	tensor.hpp, tensor.cpp
1.2	Autograd Node & Graph	node.hpp, node.cpp
1.3	Core Differentiable Ops	ops.hpp, ops.cpp
1.4	Topological Backward Pass	autograd.hpp
2.1	Module Base, Linear, LayerNorm	module.hpp, linear.*, layernorm.*
2.2	Embedding, Activation, Softmax	embedding.*, activation.*, softmax.*
2.3	★ Fused Causal Self-Attention	attention.hpp, attention.cpp
2.4	FFN, TransformerBlock, Transformer	feedforward.*, transformer_block.*, transformer.*
3.1	Python Preprocessor	scripts/preprocess.py
3.2	POSIX mmap DataLoader	dataloader.*, curriculum.hpp
4.1	Cross-Entropy Loss & AdamW	cross_entropy.*, adam.*
4.2	Curriculum Training Loop	main.cpp
5.1	GTest: Engine & NN	test_tensor.cpp ...test_attention.cpp
5.2	GTest: DataLoader + Benchmarks	test_dataloader.cpp, bench_*.cpp
5.3	CMake Build & REST API	CMakeLists.txt, http_server.*, inference_handler.*

Table 1: Complete 15-step development roadmap. Steps 1.1 and 1.2 are implemented in this report.

1.3 Repository Structure (Engine Sub-tree)

```
transformer/project/  
|  
+-- engine/                                # Core Math Engine & Autograd  
|   +-- tensor.hpp / tensor.cpp           # [Step 1.1] N-dim Tensor (DONE  
|   )  
|   +-- node.hpp / node.cpp               # [Step 1.2] Autograd Node (  
|   DONE)  
|   +-- ops.hpp / ops.cpp                 # [Step 1.3] Differentiable ops  
|   +-- autograd.hpp                     # [Step 1.4] Topological  
|   backward()  
|  
+-- nn/                                    # Neural Network Modules  
+-- data_loader/                          # POSIX mmap DataLoader  
+-- optim/                                # Optimizers  
+-- loss/                                 # Loss Functions  
+-- server/                              # REST API  
+-- tests/                               # GTest Suite  
+-- benchmarks/                          # Performance Benchmarks  
+-- scripts/preprocess.py                 # Python data preprocessor  
+-- main.cpp                             # Curriculum training entry  
point
```

Listing 1: Directory layout for the engine layer

2 Step 1.1 - Tensor Foundation

Step 1.1 Goal

Define the core N-dimensional data container used by every component in the engine. No Node, no ops, no autograd - just raw tensor mathematics.

2.1 Design Decisions

Decision	Rationale
<code>std::vector<double></code> flat 1-D storage	Single contiguous allocation maximises L1/L2 cache locality vs. nested <code>vector<vector<...>></code> .
Row-major (C-order) stride layout	Matches C++ memory layout; enables GCC/Clang SIMD auto-vectorisation under <code>-O2</code> .
Pre-computed strides at construction	Eliminates repeated stride multiplication in the hot-path <code>at()</code> indexing call.
Bounds-checked <code>at()</code>	Throws <code>std::out_of_range</code> on bad index - caught cleanly by GTest <code>EXPECT_THROW</code> .
<code>reshape()</code> validates element count	Guarantees no data copy and no reallocation - just metadata update + stride recomputation.

Table 2: Key design decisions for `engine/tensor.hpp`.

2.2 Row-Major Stride Formula

For a tensor of shape $[d_0, d_1, \dots, d_{n-1}]$, the row-major (C-order) strides are:

$$\text{strides}[n-1] = 1, \quad \text{strides}[i] = \text{strides}[i+1] \times d_{i+1} \quad \text{for } i < n-1$$

The flat 1-D index for a multi-dimensional index $(i_0, i_1, \dots, i_{n-1})$ is then:

$$\text{flat} = \sum_{k=0}^{n-1} i_k \times \text{strides}[k]$$

Interview Talking Point

Row-major layout means the *innermost* dimension is contiguous in memory. For a $[B, T, d]$ tensor (batch, sequence, features), iterating over the feature dimension d is cache-friendly - exactly the access pattern in the attention score computation $Q \cdot K^\top$.

2.3 File: engine/tensor.hpp

```

1  /**
2   * @file      engine/tensor.hpp
3   * @brief     N-dimensional Tensor with flat contiguous storage
4   *           and row-major strides.
5   *
6   * Design decisions (implementation_plan_v2.md, Step 1.1):
7   * - Storage   : flat std::vector<double> for L1/L2 cache locality.
8   * - Metadata  : shape + pre-computed row-major strides.
9   * - Indexing  : bounds-checked at() converts N-d indices to 1-D
10   *              flat index.
11   * - No Eigen, no LibTorch, no Boost. Pure C++17 STL only.
12   */
13 #pragma once
14
15 #include <cstdint>
16 #include <initializer_list>
17 #include <ostream>
18 #include <stdexcept>
19 #include <string>
20 #include <vector>
21
22 namespace engine {
23
24 class Tensor {
25 public:
26     // -- Constructors
27     -----
28
29     /** Zero-initialised tensor of given shape. */
30     explicit Tensor(std::vector<size_t> shape);
31
32     /** Construct from existing flat buffer (validates size ==
33         product(shape)). */
34     Tensor(std::vector<size_t> shape, std::vector<double> data);
35
36     /** Brace-list shape convenience: Tensor t({3, 4}); */
37     Tensor(std::initializer_list<size_t> shape);
38
39     // -- Shape & Stride Accessors
40     -----
41
42     [[nodiscard]] size_t ndim() const noexcept { return shape_.
43         size(); }
44     [[nodiscard]] size_t numel() const noexcept { return data_.size
45         (); }
46
47     [[nodiscard]] const std::vector<size_t>& shape()
48         const noexcept { return shape_; }
49     [[nodiscard]] const std::vector<size_t>& strides()
50         const noexcept { return strides_; }
51
52     // -- Bounds-Checked Element Access
53     -----
54
55     /**

```

```

50     * flat_index = sum_i ( indices[i] * strides_[i] )
51     * Throws std::out_of_range if any index exceeds its dimension
      bound.
52     * Throws std::invalid_argument if indices.size() != ndim().
53     */
54 [[nodiscard]] double& at(const std::vector<size_t>& indices);
55 [[nodiscard]] const double& at(
56     const std::vector<size_t>& indices) const;
57
58 // -- Raw Data Access
      -----
59
60 [[nodiscard]] std::vector<double>& data() noexcept;
61 [[nodiscard]] const std::vector<double>& data() const noexcept
      ;
62 [[nodiscard]] double* data_ptr() noexcept;
63 [[nodiscard]] const double* data_ptr() const noexcept;
64
65 // -- Shape Manipulation
      -----
66
67 /**
68  * Reshape in-place.
69  * Validates: product(new_shape) == numel().
70  * Recomputes strides. Data buffer is NEVER copied.
71  */
72 void reshape(std::vector<size_t> new_shape);
73
74 // -- Utility
      -----
75
76 void fill(double value) noexcept;
77 void zero() noexcept { fill(0.0); }
78
79 void print(std::ostream& os, int precision = 3) const;
80 void print(int precision = 3) const;
81
82 [[nodiscard]] std::string shape_str() const;
83
84 private:
85     /** strides_[ndim-1] = 1;
86      * strides_[i] = strides_[i+1] * shape_[i+1] */
87     void compute_strides();
88
89     /** Validates rank + per-dimension bounds, returns flat index.
90      */
91     [[nodiscard]] size_t flat_index(
92         const std::vector<size_t>& indices) const;
93
94     std::vector<size_t> shape_;    ///< e.g. {2, 3, 4}
95     std::vector<size_t> strides_;  ///< row-major, in element units
96     std::vector<double> data_;     ///< flat contiguous storage
97 };
98 } // namespace engine

```

Listing 2: engine/tensor.hpp - complete header

2.4 File: engine/tensor.cpp

```

1  #include "tensor.hpp"
2  #include <algorithm>
3  #include <numeric>
4  #include <iomanip>
5  #include <iostream>
6  #include <sstream>
7  #include <stdexcept>
8
9  namespace engine {
10
11  // -- Constructors
12  -----
13  Tensor::Tensor(std::vector<size_t> shape)
14      : shape_(std::move(shape))
15  {
16      if (shape_.empty())
17          throw std::invalid_argument(
18              "Tensor: shape must have at least one dimension.");
19      for (size_t i = 0; i < shape_.size(); ++i)
20          if (shape_[i] == 0)
21              throw std::invalid_argument(
22                  "Tensor: dimension " + std::to_string(i) + " has
23                      size 0.");
24      compute_strides();
25      data_.assign(product(shape_), 0.0);    // zero-init
26  }
27
28  // -- Row-Major Stride Computation
29  -----
30  void Tensor::compute_strides() {
31      const size_t ndim = shape_.size();
32      strides_.resize(ndim);
33      if (ndim > 0) {
34          strides_[ndim - 1] = 1;            // innermost dim stride =
35                                             1
36          for (size_t i = ndim - 1; i-- > 0; )
37              strides_[i] = strides_[i + 1] * shape_[i + 1];
38      }
39  }
40
41  // -- Bounds-Checked Flat Index Computation
42  -----
43  size_t Tensor::flat_index(const std::vector<size_t>& indices) const
44  {
45      if (indices.size() != shape_.size())
46          throw std::invalid_argument("Tensor::at(): rank mismatch.");
47      ;
48
49      size_t flat = 0;
50      for (size_t i = 0; i < indices.size(); ++i) {
51          if (indices[i] >= shape_[i])
52              throw std::out_of_range(

```

```

49         "Tensor::at(): index " + std::to_string(indices[i])
50         +
51         " out of range for dim " + std::to_string(i) +
52         " (size=" + std::to_string(shape_[i]) + ").");
53     flat += indices[i] * strides_[i]; // accumulate dot
54     product
55 }
56 return flat;
57 }
58 // -- Element Access
59 -----
60 double& Tensor::at(const std::vector<size_t>& indices) {
61     return data_[flat_index(indices)];
62 }
63 const double& Tensor::at(const std::vector<size_t>& indices) const
64 {
65     return data_[flat_index(indices)];
66 }
67 // -- Reshape
68 -----
69 void Tensor::reshape(std::vector<size_t> new_shape) {
70     if (new_shape.empty())
71         throw std::invalid_argument(
72             "Tensor::reshape(): new shape must not be empty.");
73     const size_t new_numel = product(new_shape);
74     if (new_numel != numel())
75         throw std::invalid_argument(
76             "Tensor::reshape(): size " + std::to_string(numel()) +
77             " cannot become " + std::to_string(new_numel) + ".");
78     shape_ = std::move(new_shape);
79     compute_strides(); // recompute; data_ is UNTOUCHED
80 }
81
82 } // namespace engine

```

Listing 3: engine/tensor.cpp - key implementation sections

Step 1.1 - Exit Criterion (met)

Tensor({2,3}) constructs correctly. at({1,2}) returns the element at row 1, column 2. reshape({6}) succeeds; reshape({7}) throws std::invalid_argument. print() outputs labelled rows with correct values.

3 Step 1.2 - Autograd Node & Graph

Step 1.2 Goal

Implement the `Node` class - the atomic unit of the computation DAG. A `Node` wraps a forward-pass `Tensor`, a gradient accumulator, and a backward closure. It must NOT implement any operations (Step 1.3) or the topological sort (Step 1.4).

3.1 Ownership Model & Memory Safety

The central challenge in autograd is managing a *dynamic* DAG where:

- Multiple ops may read the same parameter (fan-out).
- A backward pass must keep all intermediate nodes alive until their `_backward()` has been called.
- After backprop, memory should be released automatically.

The solution uses two smart pointer types with distinct roles:

Pointer	Where Used	Role
<code>shared_ptr<Node></code> (<code>NodePtr</code>)	User variables, lambda captures in <code>_backward</code>	Strong own-er-ship - keeps <code>Node</code> alive
<code>weak_ptr<Node></code>	<code>Node::children</code> vector	Graph traversal only - does NOT extend life-time

Table 3: Smart pointer roles in the autograd DAG.

3.1.1 Why `weak_ptr` in `children`?

Consider $c = f(a, b)$:

`c._backward` captures `shared_ptr{a}`, `shared_ptr{b}` $\Rightarrow$ a, b stay alive during backprop.

`c.children = {weak_ptr{a}, weak_ptr{b}}` $\Rightarrow$ traversal only, no extra ownership.

If `children` used `shared_ptr`, then c would own a and b , and if a or b also referenced c (e.g. in a residual connection graph), we would have a reference cycle and a memory leak. `weak_ptr` breaks this cycle structurally.

3.2 Gradient Accumulation

Because one tensor may fan out to N downstream operations, the gradient of that tensor is the *sum* of all upstream contributions:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \sum_{k=1}^N \frac{\partial \mathcal{L}}{\partial y_k} \cdot \frac{\partial y_k}{\partial \mathbf{w}}$$

`accumulate_grad()` implements this elementwise:

$$\text{grad}[i] += \text{incoming_grad}[i] \quad \forall i \in [0, \text{numel})$$

Interview Talking Point

Why accumulate (`+=`) and not assign (`=`)? In a multi-branch computation graph (e.g. a weight matrix shared by an attention layer and an FFN layer), the same parameter node receives gradient contributions from *both* branches during backprop. Over-writing (`=`) would silently discard one branch's gradient, causing incorrect parameter updates. Accumulation (`+=`) is the mathematically correct operation and mirrors what PyTorch's `autograd` engine does internally.

3.3 File: engine/node.hpp

```

1  /**
2   * @file      engine/node.hpp
3   * @brief     Autograd Node -- atomic unit of the computation DAG.
4   *
5   * Ownership: std::shared_ptr<Node> (NodePtr) for lifetime
6   *             management.
7   * Cycle prevention: children stored as std::weak_ptr<Node>.
8   * Backward lambda captures shared_ptr inputs -- the true ownership
9   *             path.
10  */
11
12  #pragma once
13
14  #include "tensor.hpp"
15  #include <functional>    // std::function
16  #include <memory>        // std::shared_ptr, std::weak_ptr
17  #include <string>
18  #include <vector>
19
20  namespace engine {
21
22  class Node;
23  using NodePtr = std::shared_ptr<Node>;
24
25  class Node {
26  public:
27      // -- Core Members (public: accessed directly by ops & autograd) ---
28
29      Tensor data;    ///< Forward-pass values. Never modified by
30                      backward.
31      Tensor grad;    ///< Gradient accumulator (same shape as data,
32                      zeros).
33
34      /**
35       * Backward closure registered by the producing op.
36       * Captures shared_ptr to each input Node.
37       * Default: no-op lambda [](){} for leaf and non-differentiable
38       * nodes.
39       */
40      std::function<void()> _backward;
41
42      /**
43       * Weak references to input nodes.
44       * Used by topological sort (Step 1.4) for DAG traversal.
45       * weak_ptr prevents reference cycles; _backward lambda owns
46       * inputs.
47       */
48      std::vector<std::weak_ptr<Node>> children;
49
50      bool requires_grad;    ///< True for trainable parameters.
51
52      // -- Constructors
53      -----
54
55      explicit Node(Tensor tensor_data, bool requires_grad = false);

```

```

49     Node(Tensor&& tensor_data,          bool requires_grad = false);
50
51     Node(const Node&)                    = delete;    // No copy -- use
        NodePtr
52     Node& operator=(const Node&) = delete;
53     Node(Node&&)                  = default;
54     Node& operator=(Node&&)       = default;
55     ~Node()                       = default;
56
57     // -- Factory (preferred construction path)
        -----
58
59     /** Single std::make_shared call: one allocation for Node +
        refcount. */
60     [[nodiscard]] static NodePtr make(Tensor tensor_data,
61                                     bool requires_grad = false
62                                     );
63     [[nodiscard]] static NodePtr make(Tensor&& tensor_data,
64                                     bool requires_grad =
65                                     false);
66
67     // -- Graph Construction
        -----
68
69     /** Store child as weak_ptr. Called by ops when building the
        DAG. */
70     void add_child(const NodePtr& child);
71
72     // -- Gradient Helpers
        -----
73
74     /**
75      * Elementwise: grad[i] += incoming_grad[i]
76      * Operates on raw data_ptr() buffers -- no new graph nodes.
77      * @throws std::invalid_argument if shapes differ.
78      */
79     void accumulate_grad(const Tensor& incoming_grad);
80
81     /** Reset grad to all zeros. Called before each new backward
        pass. */
82     void zero_grad() noexcept;
83
84     // -- Introspection
        -----
85
86     [[nodiscard]] bool is_leaf() const noexcept { return children.
        empty(); }
87
88     /** Print shape, requires_grad, is_leaf, child count, grad L2
        norm. */
89     void print_info() const;
90 };
91
92 // -- Free-function convenience wrappers
        -----
93
94 /** Leaf parameter node (requires_grad = true). */
95 [[nodiscard]] inline NodePtr make_parameter(Tensor t) {

```

```
94     return Node::make(std::move(t), true);
95 }
96
97 /** Non-differentiable data node (requires_grad = false). */
98 [[nodiscard]] inline NodePtr make_input(Tensor t) {
99     return Node::make(std::move(t), false);
100 }
101
102 } // namespace engine
```

Listing 4: engine/node.hpp - complete header

3.4 File: engine/node.cpp

```

1  #include "node.hpp"
2  #include <cmath>           // std::sqrt (for grad L2 norm in
    print_info)
3  #include <iostream>
4  #include <stdexcept>
5
6  namespace engine {
7
8  // -- Constructors
    -----
9
10 Node::Node(Tensor tensor_data, bool req_grad)
11     : data(std::move(tensor_data))
12     , grad(data.shape())           // same shape as data, zero-
    initialised
13     , _backward([]() noexcept {}) // default: no-op backward
14     , children()                   // empty -- leaf node
15     , requires_grad(req_grad)
16 {}
17
18 // -- Factory
    -----
19
20 NodePtr Node::make(Tensor tensor_data, bool requires_grad) {
21     // std::make_shared: one allocation for Node + shared_ptr
    control block.
22     // Better cache behaviour than: shared_ptr<Node>(new Node(...))
23     return std::make_shared<Node>(std::move(tensor_data),
    requires_grad);
24 }
25
26 // -- Graph Construction
    -----
27
28 void Node::add_child(const NodePtr& child) {
29     // Implicit NodePtr --> weak_ptr<Node> conversion.
30     // The topological sort (Step 1.4) will call lock() on each
    weak_ptr
31     // to obtain a temporary shared_ptr during DAG traversal.
32     children.emplace_back(child);
33 }
34
35 // -- Gradient Accumulation
    -----
36
37 void Node::accumulate_grad(const Tensor& incoming_grad) {
38     // Shape guard
39     if (grad.shape() != incoming_grad.shape())
40         throw std::invalid_argument(
41             "Node::accumulate_grad: shape mismatch -- grad=" +
42             grad.shape_str() + " incoming=" + incoming_grad.
    shape_str());
43
44     // Elementwise grad[i] += incoming_grad[i]
45     // Operates directly on flat data_ptr() buffers.
46     // - No autograd graph entries are created.

```

```

47 // - Loop is trivially SIMD-vectorisable under -O2.
48 // - No dependency on ops.hpp (Step 1.2 stays self-contained).
49 const size_t n = grad.numel();
50 double* dst = grad.data_ptr();
51 const double* src = incoming_grad.data_ptr();
52
53 for (size_t i = 0; i < n; ++i)
54     dst[i] += src[i];
55 }
56
57 void Node::zero_grad() noexcept {
58     grad.zero(); // Tensor::zero() calls fill(0.0) on the flat
59                 // buffer
60 }
61 // -- Introspection
62 -----
63 void Node::print_info() const {
64     // Compute L2 norm of gradient for debugging convergence
65     const double* p = grad.data_ptr();
66     double sum = 0.0;
67     for (size_t i = 0; i < grad.numel(); ++i) sum += p[i] * p[i];
68     const double norm = std::sqrt(sum);
69
70     std::cout << "Node {\n"
71               << "  data.shape      = " << data.shape_str()
72               << "\n"
73               << "  requires_grad = " << (requires_grad ? "true": "
74               << "false") << "\n"
75               << "  is_leaf      = " << (is_leaf() ? "true": "
76               << "false") << "\n"
77               << "  num_children = " << children.size()
78               << "\n"
79               << "  grad_l2_norm = " << norm
80               << "\n"
81               << "  has_backward = " << (_backward ? "yes": "no (
82               << "null)") << "\n"
83               << "}\n";
84 }
85 } // namespace engine

```

Listing 5: engine/node.cpp - key implementation sections

Step 1.2 - Exit Criterion (met)

Node::make(Tensor({4,8}), true) creates a heap-allocated parameter node. grad is zeroed and matches data.shape(). Two nodes linked via add_child() - parent's children holds a weak_ptr to the child. Manually calling _backward() executes the no-op without crash. accumulate_grad() correctly computes grad += incoming elementwise.

4 Key Design Patterns (Interview Reference)

4.1 Smart Pointer Ownership Pattern

The ownership graph for $c = \text{matmul}(a, b)$ after Step 1.3 will be:

Reference	Type	Points To
User variable <code>a</code>	<code>shared_ptr</code>	Node A
User variable <code>b</code>	<code>shared_ptr</code>	Node B
User variable <code>c</code>	<code>shared_ptr</code>	Node C
<code>c._backward</code> closure (capture of <code>a</code>)	<code>shared_ptr</code>	Node A
<code>c._backward</code> closure (capture of <code>b</code>)	<code>shared_ptr</code>	Node B
<code>c.children[0]</code>	<code>weak_ptr</code>	Node A
<code>c.children[1]</code>	<code>weak_ptr</code>	Node B

When the user drops `c`, all `shared_ptr` references to `C` reach zero and `C` is freed. The lambda is destroyed, releasing the `shared_ptr` captures of `A` and `B`. If the user also drops `a` and `b`, all nodes are freed - no leak, no dangling pointer.

4.2 Gradient Accumulation vs. Assignment

	Assignment (=)	Accumulation (+=)
Single use	Correct	Correct
Fan-out (shared weight)	WRONG - drops all but last gradient	Correct - sums all contributions
Usage	Never correct in autograd	Always required

Table 4: Why gradient *accumulation* is mandatory.

4.3 Memory Layout - Cache Locality Impact

5 Build & Verification Instructions

5.1 Compile Steps 1.1 + 1.2 (Standalone Smoke Test)

```

1 # Compile both translation units together
2 g++ -std=c++17 -O2 -Wall -Wextra -o smoke_test \
3     engine/tensor.cpp \
4     engine/node.cpp \
5     smoke_test.cpp

```

Layout	Structure	Cache Behaviour
Flat 1-D <code>vector<double></code> (ours)	[a00, a01, a02, a10, a11, a12]	Sequential reads in all inner-loop ops - L1 hits
Nested <code>vector<vector<double></code>	Each row is a separate heap allocation	Pointer chasing between rows - L1/L2 misses

Table 5: Why flat 1-D storage is critical for performance.

```

6
7 # Run
8 ./smoke_test

```

Listing 6: Build commands (WSL2 / Linux)

5.2 Minimal Smoke Test (smoke_test.cpp)

```

1 #include "engine/tensor.hpp"
2 #include "engine/node.hpp"
3 #include <cassert>
4 #include <iostream>
5
6 int main() {
7     using namespace engine;
8
9     // -- Step 1.1: Tensor checks
10    -----
11    Tensor t({2, 3}, {1,2,3,4,5,6});
12    assert(t.numel() == 6);
13    assert(t.ndim() == 2);
14    assert(t.at({0, 0}) == 1.0);
15    assert(t.at({1, 2}) == 6.0);
16    assert(t.strides()[0] == 3); // row-major: stride[0] = cols
17    assert(t.strides()[1] == 1); // innermost stride = 1
18
19    t.reshape({6}); // valid -- same numel
20    assert(t.ndim() == 1);
21
22    try {
23        t.reshape({7}); // invalid -- numel mismatch
24        assert(false && "should have thrown");
25    } catch (const std::invalid_argument&) { /* expected */ }
26
27    t.print();
28
29    // -- Step 1.2: Node checks
30    -----
31    auto w = make_parameter(Tensor({2,3}, {1,2,3,4,5,6}));
32    assert(w->requires_grad == true);
33    assert(w->is_leaf() == true);
34    assert(w->grad.numel() == 6);
35
36    // Accumulate a gradient contribution

```

```

35     Tensor g({2,3}, {0.1, 0.2, 0.3, 0.4, 0.5, 0.6});
36     w->accumulate_grad(g);
37     assert(w->grad.at({0,0}) == 0.1);
38     assert(w->grad.at({1,2}) == 0.6);
39
40     // Accumulate again (fan-out simulation)
41     w->accumulate_grad(g);
42     assert(w->grad.at({0,0}) == 0.2); // 0.1 + 0.1
43
44     w->zero_grad();
45     assert(w->grad.at({0,0}) == 0.0);
46
47     w->print_info();
48
49     std::cout << "All assertions passed -- Steps 1.1 and 1.2
50     verified.\n";
51     return 0;
52 }

```

Listing 7: smoke_test.cpp - verifies Steps 1.1 and 1.2

6 What Comes Next

Step	What Will Be Built
1.3	ops.hpp / ops.cpp - differentiable operations (add, mul, matmul, exp, log, sum, transpose). Each op creates a new Node, sets <code>_backward</code> , and calls <code>add_child()</code> on each input.
1.4	autograd.hpp - <code>backward(Node& root)</code> : DFS post-order topological sort over the DAG; sets <code>root.grad = 1.0</code> ; calls <code>_backward()</code> in reverse topological order.
2.1	nn/module.hpp, nn/linear.*, nn/layernorm.* - first neural network modules.
2.3	★ Fused Causal Self-Attention - <code>CausalSelfAttention</code> with <code>W_Q</code> , <code>W_K</code> , <code>W_V</code> , <code>W_O</code> projections, scaled dot-product, and lower-triangular causal mask.

Table 6: Upcoming implementation steps.

7 Step 1.3 - Core Differentiable Tensor Operations

Step 1.3 Goal

Implement the **7 free functions** (in `engine::ops`) that populate the autograd DAG during the forward pass and register the backward lambda that drives gradient computation. Every operation must satisfy a strict 4-step autograd contract. No topological sort yet (Step 1.4).

7.1 The Autograd Contract

Every function in `engine::ops` must:

1. **Forward**: compute the output into a new `Tensor`.
2. **Wrap**: create a `NodePtr` with `Node::make(...)`.
3. **DAG edges**: call `out->add_child(input)` for every input.
4. **Backward**: assign a lambda to `out->_backward` that applies the chain rule and calls `input->accumulate_grad(...)`.

7.2 Critical Design: Cycle-Safe Backward Lambda Captures

This is the most subtle correctness issue in the entire autograd implementation.

7.2.1 The Cycle Problem

```
// WRONG -- creates a memory leak
out->_backward = [a, b, out]() {           // out is shared_ptr<Node>
    >
    a->accumulate_grad(out->grad);          // out->_backward -->
    out
    b->accumulate_grad(out->grad);          // Node C owns lambda
};                                           // lambda owns Node C
--> CYCLE
```

Listing 8: Naive capture creates a self-reference cycle

When the user drops their `NodePtr` to the output, the reference count stays at 1 (the lambda still holds it). The lambda cannot be destroyed because it lives inside the Node. **Result: memory leak.**

7.2.2 The Fix: `weak_ptr` Self-Capture

```
1 // CORRECT -- wout does NOT keep Node C alive
2 out->_backward = [a, b, wout = std::weak_ptr<Node>(out)]() {
3     auto self = wout.lock();           // obtain temporary shared_ptr
4     if (!self) return;                 // safety check (always valid during
    backward)
5     a->accumulate_grad(self->grad);
6     b->accumulate_grad(self->grad);
7 };
```

Listing 9: Correct cycle-safe capture pattern used in all ops

Interview Talking Point - Why is `weak_ptr` safe here?

During `backward()` (Step 1.4), the topological sort holds a `std::vector<NodePtr>` of *all* reachable nodes. This vector keeps every intermediate node alive for the full duration of the backward pass, so `wout.lock()` is always valid when the lambda executes. After `backward()` returns, the vector goes out of scope, all intermediate nodes are freed simultaneously, and no leak is possible.

7.3 High-Performance Tiled MatMul

The `matmul` op is the most performance-critical function in the engine. It uses three cache-blocked kernels:

Kernel	Computes	Used For
<code>mm_nn</code>	$C += A \cdot B$	Forward pass
<code>mm_nt</code>	$C += A \cdot B^T$	Backward: $\partial L / \partial A$
<code>mm_tn</code>	$C += A^T \cdot B$	Backward: $\partial L / \partial B$

Table 7: Three tiled matmul kernels and their roles.

7.3.1 Tiling Algorithm (`mm_nn`)

`BLOCK_SIZE = 64 doubles = 512 bytes per tile row`

One tile = 64×64 doubles = 32 KB $\leq$ typical L1 cache (32–64 KB)

The loop nest orders iterations as (ii, jj, kk, i, k, j) :

- Hoisting `a_ik = A[i*K+k]` out of the inner `j`-loop eliminates one memory load per iteration.
- Sequential `j`-access of `B[k*N+j]` enables hardware prefetch and SIMD auto-vectorisation by GCC/Clang at `-O2`.
- `#pragma omp parallel for` on the outermost `ii` loop: each thread owns complete output tile rows, so no write conflicts on `C`.

```

1 static constexpr size_t BLOCK_SIZE = 64;
2
3 void mm_nn(const double* A, const double* B, double* C,
4           size_t M, size_t K, size_t N)
5 {
6     #pragma omp parallel for schedule(dynamic, 4)
7     for (size_t ii = 0; ii < M; ii += BLOCK_SIZE) {
8         for (size_t jj = 0; jj < N; jj += BLOCK_SIZE) {
9             for (size_t kk = 0; kk < K; kk += BLOCK_SIZE) {
10                 const size_t imax = std::min(ii + BLOCK_SIZE, M);

```

```

11         const size_t jmax = std::min(jj + BLOCK_SIZE, N);
12         const size_t kmax = std::min(kk + BLOCK_SIZE, K);
13         for (size_t i = ii; i < imax; ++i) {
14             for (size_t k = kk; k < kmax; ++k) {
15                 const double a_ik = A[i * K + k];    //
16                 register hoist
17                 for (size_t j = jj; j < jmax; ++j)
18                     C[i * N + j] += a_ik * B[k * N + j];
19             }
20         }
21     }
22 }
23 }

```

Listing 10: mm_nn - cache-blocked tiled GEMM kernel

7.4 Backward Derivations for All 7 Ops

Op	Forward	Backward (chain rule)
add	$z_i = a_i + b_i$	$\partial a_i = \partial z_i, \quad \partial b_i = \partial z_i$
mul	$z_i = a_i \cdot b_i$	$\partial a_i = \partial z_i \cdot b_i, \quad \partial b_i = \partial z_i \cdot a_i$
matmul	$Z = AB$	$\partial A = \partial Z \cdot B^\top, \quad \partial B = A^\top \cdot \partial Z$
exp	$z_i = e^{a_i}$	$\partial a_i = \partial z_i \cdot e^{a_i} = \partial z_i \cdot z_i$
log	$z_i = \ln a_i$	$\partial a_i = \partial z_i / a_i$
sum	$z = \sum_i a_i$	$\partial a_i = \partial z$ (broadcast)
transpose	$z_{ij} = a_{ji}$	$\partial a_{ji} = \partial z_{ij}$ (transpose the gradient)

Table 8: Forward computations and chain-rule backward formulas for all 7 ops.

7.5 File: engine/ops.hpp

```

1  /**
2   * @file engine/ops.hpp
3   * @brief Differentiable tensor operations - the DAG-building layer
4   *
5   * Every op follows the autograd contract:
6   * 1. Compute forward result.
7   * 2. Wrap in a new NodePtr.
8   * 3. Register DAG edges via add_child().
9   * 4. Assign _backward lambda with chain-rule + weak_ptr self-
10     capture.
11  */
12 #pragma once
13 #include "node.hpp"
14

```

```

15 namespace engine::ops {
16
17 // -- Binary ops
18 -----
19 [[nodiscard]] NodePtr add      (const NodePtr& a, const NodePtr& b)
20 ;
21 [[nodiscard]] NodePtr mul      (const NodePtr& a, const NodePtr& b)
22 ;
23 [[nodiscard]] NodePtr matmul   (const NodePtr& a, const NodePtr& b)
24 ;
25
26 // -- Unary elementwise ops
27 -----
28 [[nodiscard]] NodePtr exp      (const NodePtr& a);
29 [[nodiscard]] NodePtr log      (const NodePtr& a);
30
31 // -- Reduction ops
32 -----
33 [[nodiscard]] NodePtr sum      (const NodePtr& a);
34
35 // -- Shape ops
36 -----
37 [[nodiscard]] NodePtr transpose(const NodePtr& a);
38
39 } // namespace engine::ops

```

Listing 11: engine/ops.hpp - complete header

7.6 File: engine/ops.cpp - Selected Implementations

```

1  NodePtr add(const NodePtr& a, const NodePtr& b) {
2      // Step 1: Shape validation + forward computation
3      if (a->data.shape() != b->data.shape())
4          throw std::invalid_argument("ops::add: shape mismatch");
5
6      const size_t n = a->data.numel();
7      std::vector<double> fwd(n);
8      for (size_t i = 0; i < n; ++i)
9          fwd[i] = a->data.data_ptr()[i] + b->data.data_ptr()[i];
10
11     // Step 2: Create output node (requires_grad propagated from
12     // inputs)
13     auto out = Node::make(
14         Tensor(a->data.shape(), std::move(fwd)),
15         a->requires_grad || b->requires_grad
16     );
17
18     // Step 3: DAG edges
19     out->add_child(a);
20     out->add_child(b);
21
22     // Step 4: Backward lambda (wout = weak_ptr breaks self-ref
23     // cycle)
24     out->_backward = [a, b, wout = std::weak_ptr<Node>(out)]() {
25         auto self = wout.lock();
26         if (!self) return;
27         // add is linear: local gradient is 1.0 -> pass-through

```

```

26     a->accumulate_grad(self->grad);
27     b->accumulate_grad(self->grad);
28 };
29
30     return out;
31 }

```

Listing 12: add - simplest op, shows the full 4-step contract

```

1  // Backward: called by topological sort in Step 1.4
2  out->_backward = [a, b, M, K, N, batch,
3                    wout = std::weak_ptr<Node>(out)]() {
4      auto self = wout.lock();
5      if (!self) return;
6
7      const double* dout = self->grad.data_ptr();
8      Tensor da(a->data.shape());    // zero-init
9      Tensor db(b->data.shape());    // zero-init
10
11     for (size_t batch_i = 0; batch_i < batch; ++batch_i) {
12         const double* dout_b = dout + batch_i * M * N;
13         const double* a_b     = a->data.data_ptr() + batch_i * M * K
14             ;
15         const double* b_b     = b->data.data_ptr() + batch_i * K * N
16             ;
17
18         // dA[M,K] += dOut[M,N] @ B^T[N,K] (mm_nt kernel, tiled+
19             OpenMP)
20         mm_nt(dout_b, b_b, da.data_ptr() + batch_i*M*K, M, N, K);
21
22         // dB[K,N] += A^T[K,M] @ dOut[M,N] (mm_tn kernel, tiled+
23             OpenMP)
24         mm_tn(a_b, dout_b, db.data_ptr() + batch_i*K*N, M, K, N);
25     }
26
27     a->accumulate_grad(da);
28     b->accumulate_grad(db);
29 };

```

Listing 13: matmul backward - tiled kernels for dA and dB

```

1  NodePtr exp(const NodePtr& a) {
2      // Forward: out[i] = exp(a[i])
3      std::vector<double> fwd(a->data.numel());
4      for (size_t i = 0; i < fwd.size(); ++i)
5          fwd[i] = std::exp(a->data.data_ptr()[i]);
6
7      auto out = Node::make(Tensor(a->data.shape(), std::move(fwd)),
8                            a->requires_grad);
9      out->add_child(a);
10
11     // Backward: d/dx exp(x) = exp(x) = out->data[i]
12     // Key insight: we reuse the already-computed out->data
13     // -- no need to call exp() again in the backward pass.
14     out->_backward = [a, wout = std::weak_ptr<Node>(out)]() {
15         auto self = wout.lock();
16         if (!self) return;
17         Tensor da(a->data.shape());

```

```

18     const double* dout  = self->grad.data_ptr();
19     const double* out_d = self->data.data_ptr(); // stored exp
        values
20     double*      dp      = da.data_ptr();
21     for (size_t i = 0; i < da.numel(); ++i)
22         dp[i] = dout[i] * out_d[i]; // chain rule: dL/dout *
        exp(a)
23     a->accumulate_grad(da);
24 };
25 return out;
26 }

```

Listing 14: `exp` - reuses forward output in backward**Step 1.3 - Exit Criterion (met)**

`c = ops::matmul(a, b)` produces correct shape. After `c->_backward()`, `a->grad` and `b->grad` contain the correct gradients (verified numerically in Step 1.4's grad-check). After dropping all user-side `NodePtr` variables, Valgrind reports zero memory leaks - the `weak_ptr` self-capture eliminates the self-reference cycle.

8 Step 1.4 - Topological Backward Pass

Step 1.4 Goal

Implement `backward(root)` - the final piece of the core autograd engine. This function seeds the scalar loss gradient, runs a DFS post-order topological sort over the computation DAG, then calls every node's `_backward()` in **reverse topological order**, propagating gradients from the loss all the way back to the leaf parameters.

8.1 Algorithm: DFS Post-Order Topological Sort

8.1.1 Why Post-Order?

The DAG has edges flowing from output nodes to their input nodes (set by `add_child(input)` in `ops.cpp`). For backward propagation to be correct, we need the following invariant:

Invariant: A node's gradient must be *fully accumulated* before its `_backward()` fires.

Example: Weight matrix $\mathbf{W}$ is used by two linear layers ($y_1 = xW$ and $y_2 = zW$). Both y_1 and y_2 must call `W.accumulate_grad()` before `W._backward()` is invoked. Post-order DFS guarantees exactly this: the sort visits W only after visiting every node that points to W as a child.

8.1.2 Post-Order DFS Trace

For a graph $a \rightarrow c \rightarrow \text{loss}$ and $b \rightarrow c$:

1. Start at `loss`. Visit child `c`.
2. At `c`. Visit child `a`. At `a`: leaf, no children **append a**.
3. Back at `c`. Visit child `b`. At `b`: leaf **append b**.
4. Back at `c`: all children done **append c**.
5. Back at `loss`: child done **append loss**.

Result: `topo = [a, b, c, loss]`

Reversed: `[loss, c, b, a]` - backward order. ✓

8.2 Visited Set: `std::unordered_set<const Node*>`

Using raw `Node*` (pointer identity) rather than `shared_ptr<Node>` as the key:

Property	Explanation
O(1) lookup	Raw pointer hash is a single integer operation.
Correct identity semantics	Two <code>shared_ptr</code> referring to the same <code>Node</code> must be treated as one visit. Pointer equality is the right comparison.
No ownership extension	Using raw pointers does NOT affect reference counts. The <code>topo</code> vector already owns a <code>shared_ptr</code> to each visited node, keeping them alive throughout the sort.
No dangling risk	As long as <code>topo</code> is in scope, all raw pointers in <code>visited</code> remain valid.

Table 9: Why `unordered_set<const Node*>` is the correct visited-set type.

8.3 Handling `weak_ptr` Children

Each `Node::children` entry is a `std::weak_ptr<Node>`. During traversal, every child is locked:

```

1 for (const std::weak_ptr<Node>& weak_child : node->children) {
2     if (NodePtr child = weak_child.lock()) {
3         dfs_postorder(child, topo, visited);
4     }
5     // null lock: node already freed -- silently skip
6 }
```

Listing 15: Safe `weak_ptr` locking in `dfs_postorder`

A null lock result is safe to skip because:

- If a node has been freed before the traversal reaches it, it has no live gradient state to propagate.
- In a well-formed training step, no node is freed during the backward pass because `topo` holds `shared_ptr` to every discovered node.

8.4 File: engine/autograd.hpp

```
1  /**
2   * @file      engine/autograd.hpp
3   * @brief     Reverse-mode automatic differentiation - backward pass.
4   *
5   * backward(root):
6   *   1. Validate scalar root.
7   *   2. Seed root->grad = 1.0.
8   *   3. DFS post-order topological sort (visited: unordered_set<
9   *      Node*>).
10  *   4. Reverse-order _backward() sweep.
11  *
12  * zero_grad_all(root):
13  *   Traverse DAG, call Node::zero_grad() on every node.
14  */
15 #pragma once
16 #include "node.hpp"
17
18 namespace engine {
19
20 /**
21  * Compute gradients for the full DAG rooted at 'root'.
22  * root must be a scalar node (numel == 1).
23  * Gradients are ACCUMULATED -- call zero_grad_all() before each
24  * step.
25  */
26 void backward(const NodePtr& root);
27
28 /**
29  * Zero grad tensors on every node reachable from 'root'.
30  */
31 void zero_grad_all(const NodePtr& root);
32 } // namespace engine
```

Listing 16: engine/autograd.hpp - complete header

8.5 File: engine/autograd.cpp

```

1 namespace {
2
3 void dfs_postorder(const NodePtr& node,
4                   std::vector<NodePtr>& topo,
5                   std::unordered_set<const Node*>& visited)
6 {
7     if (!node) return;
8
9     const Node* raw = node.get();
10    if (visited.count(raw)) return;    // already processed (DAG
11                                       fan-out)
12    visited.insert(raw);
13
14    // Recurse into children (inputs) FIRST -- post-order
15    for (const std::weak_ptr<Node>& weak_child : node->children) {
16        if (NodePtr child = weak_child.lock()) {
17            dfs_postorder(child, topo, visited);
18        }
19    }
20
21    topo.push_back(node);    // append AFTER all children -- post-
22                             order
23 } // anonymous namespace

```

Listing 17: dfs_postorder - the recursive sort helper

```

1 void backward(const NodePtr& root)
2 {
3     // -- Guard: null and non-scalar detection
4     -----
5     if (!root)
6         throw std::invalid_argument("backward(): root NodePtr is
7                                     null.");
8     if (root->data.numel() != 1)
9         throw std::invalid_argument(
10            "backward(): root must be a scalar (numel==1). "
11            "Got shape " + root->data.shape_str() + ".");
12
13    // -- Step 1: Seed gradient (dL/dL = 1.0)
14    -----
15    root->grad.data_ptr()[0] = 1.0;
16
17    // -- Step 2: Topological sort
18    -----
19    // Result: topo = [leaves ... root]
20    std::vector<NodePtr> topo;
21    std::unordered_set<const Node*> visited;
22    topo.reserve(64);
23    visited.reserve(64);
24    dfs_postorder(root, topo, visited);
25
26    // -- Step 3: Reverse sweep -- root first, leaves last
27    -----
28    // topo.rbegin() points to root, topo.rend() points past the
29    // first leaf.

```

```

24     // Each node's grad is fully accumulated (from Step 2 ordering)
        before
25     // its _backward() fires and propagates it to its children.
26     for (auto it = topo.rbegin(); it != topo.rend(); ++it) {
27         (*it)->_backward();    // no-op for leaf nodes (correct &
            harmless)
28     }
29
30     // 'topo' destroyed here -- all intermediate nodes potentially
        freed.
31 }    // weak_ptr wout.lock() in ops lambdas is safe throughout this
        scope.

```

Listing 18: backward() - the complete implementation

8.6 Correctness: The Gradient Seeding Invariant

The chain rule for the full DAG is:

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_{\text{paths } P \text{ from } \mathcal{L} \text{ to } w} \prod_{(u,v) \in P} \frac{\partial u}{\partial v}$$

backward() computes this via *dynamic programming*: each node computes its local Jacobian-vector product and accumulates it into its inputs. The topological ordering ensures the upstream product $\partial \mathcal{L} / \partial u$ is complete in u->grad before u->_backward() runs.

8.7 End-to-End Smoke Test

```

1  #include "engine/ops.hpp"
2  #include "engine/autograd.hpp"
3  #include <cassert>
4  #include <cmath>
5
6  int main() {
7      using namespace engine;
8      namespace ops = engine::ops;
9
10     // Leaf parameters: a=3.0, b=2.0
11     auto a = make_parameter(Tensor({1}, {3.0}));
12     auto b = make_parameter(Tensor({1}, {2.0}));
13
14     // Forward: c = a * b = 6.0
15     auto c = ops::mul(a, b);
16
17     // Forward: loss = c * c = 36.0 (simulate squaring)
18     auto loss = ops::mul(c, c);
19
20     // Backward
21     backward(loss);
22
23     // Analytical gradients:
24     // d(loss)/da = d((ab)^2)/da = 2*(ab)*b = 2*6*2 = 24.0
25     // d(loss)/db = d((ab)^2)/db = 2*(ab)*a = 2*6*3 = 36.0
26     assert(std::abs(a->grad.data_ptr()[0] - 24.0) < 1e-10);

```

```

27     assert(std::abs(b->grad.data_ptr()[0] - 36.0) < 1e-10);
28
29     return 0;    // All assertions passed
30 }

```

Listing 19: Full engine test: $a = 3.0$, $b = 2.0$, $\text{loss} = (a*b)^2$

Step 1.4 - Exit Criterion (met) & Phase 1 Complete

backward(loss) correctly propagates gradients through a multi-node DAG. Analytical vs. numerical gradient agreement is verified to $< 10^{-6}$ relative error. The **unordered_set** visited check correctly handles fan-out (shared weight nodes). All intermediate nodes are freed after **backward()** returns - Valgrind reports zero leaks.

Phase 1 (Steps 1.1–1.4) is now complete. The engine can build arbitrary computation graphs, perform forward passes, and propagate gradients to leaf parameters. Phase 2 (Neural Network Modules) begins with Step 2.1.

9 Step 2.1 - Module Base, Linear, and LayerNorm

Step 2.1 Goal

Build the neural network layer infrastructure. Three deliverables: **Module** (abstract base), **Linear** ($y = xW^T + b$), and **LayerNorm** (per-row normalisation with learnable γ, β). All layers live in the `engine::nn` namespace and use the Phase 1 autograd engine for all trainable parameters.

9.1 Module Base Class (nn/module.hpp)

```

1  namespace engine::nn {
2
3  class Module {
4  public:
5      virtual ~Module() = default;
6      Module(Module&&)          = default;    // move OK
7      Module(const Module&)      = delete;     // NO copy -- avoids
           param aliasing
8
9      // Every concrete module MUST list all its trainable NodePtrs
           here.
10     // Composite modules call sub-module.parameters() and
           concatenate.
11     [[nodiscard]] virtual std::vector<NodePtr> parameters() const =
           0;
12
13     // Zero-grad all parameters before each training step.
14     void zero_grad() {
15         for (const NodePtr& p : parameters()) p->zero_grad();
16     }
17 };

```

```

18
19 } // namespace engine::nn

```

Listing 20: nn/module.hpp - abstract base, header-only

Interview: Why is parameters() pure virtual?

Forcing every concrete layer to implement `parameters()` guarantees that no weight matrix is accidentally omitted from the optimizer’s update loop. For composite modules (`TransformerBlock`, `GPT`), the implementation is simply: call each sub-module’s `parameters()` and concatenate the results - giving automatic recursive collection without any registration bookkeeping.

9.2 Linear Layer

9.2.1 Forward Pass

$$y = xW^T + b$$

Tensor	Shape	Notes
weight	$[F_{\text{out}}, F_{\text{in}}]$	Xavier uniform init
bias	$[F_{\text{out}}]$	Zero init
Input x	$[B, F_{\text{in}}]$ or $[B, T, F_{\text{in}}]$	2-D or 3-D
Output y	$[\dots, F_{\text{out}}]$	same rank as x

Table 10: Linear layer tensor shapes.

The forward pass decomposes into three fully-differentiable DAG nodes:

1. $\text{Wt} = \text{ops::transpose}(\text{weight}) \rightarrow [F_{\text{in}}, F_{\text{out}}]$
Backward: $\partial W = \text{transpose}(\partial W^T)$
2. $\text{xWt} = \text{ops::matmul}(x, \text{Wt}) \rightarrow [\dots, F_{\text{out}}]$
Backward: tiled `mm_nt` / `mm_tn` kernels (Step 1.3)
3. $\text{out} = \text{bias_add}(\text{xWt}, \text{bias})$ - file-local broadcast-add op
Backward: $\partial(xW^T) = \text{pass-through}$; $\partial b[j] = \sum_i \partial y[i, j]$

9.2.2 Why a Custom bias_add Op?

`ops::add` requires identical shapes. Bias has shape $[F_{\text{out}}]$ but the matmul output has shape $[\dots, F_{\text{out}}]$. Rather than modifying the core ops layer, we implement a file-local differentiable op in `linear.cpp`:

```

1  NodePtr bias_add(const NodePtr& x, const NodePtr& b) {
2      // Forward: out[... , j] = x[... , j] + b[j]
3      ...
4      out->_backward = [x, b, N, rows, wout = std::weak_ptr<Node>(out
5                          )]() {
6          auto self = wout.lock(); if (!self) return;
7          // dx: pass-through (add is linear)

```

```

7         x->accumulate_grad(self->grad);
8         // db[j] = sum_i dout[i, j] (reduce over all leading dims)
9         Tensor db(b->data.shape());
10        for (size_t i = 0; i < rows; ++i)
11            for (size_t j = 0; j < N; ++j)
12                db.data_ptr()[j] += self->grad.data_ptr()[i*N + j];
13        b->accumulate_grad(db);
14    };
15    return out;
16 }

```

Listing 21: bias_add - broadcast-add with correct gradient reduction

9.2.3 Xavier/Glorot Uniform Initialisation

$$\text{bound} = \sqrt{\frac{6}{F_{\text{in}} + F_{\text{out}}}}, \quad W \sim \mathcal{U}(-\text{bound}, +\text{bound})$$

This keeps activation variance constant across layers, preventing vanishing or exploding gradients at initialisation for linear (or near-linear) activations.

```

1  std::mt19937 rng(std::random_device{}());
2  const double bound = std::sqrt(6.0 / (in_features + out_features));
3  std::uniform_real_distribution<double> dist(-bound, bound);
4
5  std::vector<double> w_data(out_features * in_features);
6  for (double& v : w_data) v = dist(rng);
7
8  weight = make_parameter(Tensor({out_features, in_features}, std::
    move(w_data)));
9  bias    = make_parameter(Tensor({out_features},
10                             std::vector<double>(out_features,
    0.0)));

```

Listing 22: Weight init in Linear constructor

9.3 Layer Normalization

9.3.1 Forward Formula

For input x of shape $[*, N]$ ($N = \text{d_model}$):

$$\begin{aligned} \mu_i &= \frac{1}{N} \sum_{j=0}^{N-1} x_{ij}, & \sigma_i^2 &= \frac{1}{N} \sum_{j=0}^{N-1} (x_{ij} - \mu_i)^2 \\ r_i &= \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}}, & \hat{x}_{ij} &= (x_{ij} - \mu_i) \cdot r_i \\ y_{ij} &= \gamma_j \cdot \hat{x}_{ij} + \beta_j \end{aligned}$$

9.3.2 Backward Derivation (Interview-Critical)

Let $\partial Y_{ij} = \partial \mathcal{L} / \partial y_{ij}$.

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_i \partial Y_{ij} \quad \frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_i \partial Y_{ij} \cdot \hat{x}_{ij}$$

Define per-row scalars:

$$A_i = \sum_k \gamma_k \cdot \partial Y_{ik}, \quad B_i = \sum_k \gamma_k \cdot \partial Y_{ik} \cdot \hat{x}_{ik}$$

Then:

$$\frac{\partial \mathcal{L}}{\partial x_{ij}} = r_i \left(\gamma_j \partial Y_{ij} - \frac{A_i}{N} - \hat{x}_{ij} \frac{B_i}{N} \right)$$

Interview: Why a fused op and not composed primitives?

Composing `ops::sum + ops::mul + ops::add` for LayerNorm would require a per-dim reduce-sum op (not in Step 1.3). More importantly, the fused backward is **numerically superior**: it avoids accumulating floating-point error across multiple graph nodes and saves only 2 buffers ($\hat{x}$ and r) instead of materialising every intermediate tensor. This is the same reason cuDNN provides a fused LayerNorm backward kernel.

```

1 out->_backward = [x, gamma, beta, M, N, inv_N,
2                   x_hat = std::move(x_hat_buf), // saved x_hat;
3                   lambda owns
4                   inv_std = std::move(inv_std_buf), // saved 1/
5                   sigma; lambda owns
6                   wout = std::weak_ptr<Node>(out)]()
7 {
8     auto self = wout.lock(); if (!self) return;
9     const double* dY = self->grad.data_ptr();
10    const double* gd = gamma->data.data_ptr();
11
12    Tensor dx(x->data.shape()), dgamma(gamma->data.shape()), dbeta(beta->data.shape());
13    for (size_t i = 0; i < M; ++i) {
14        // Compute A_i, B_i
15        double Ai = 0.0, Bi = 0.0;
16        for (size_t j = 0; j < N; ++j) {
17            double g_dy = gd[j] * dY[i*N+j];
18            Ai += g_dy; Bi += g_dy * x_hat[i*N+j];
19        }
20        // dx, dgamma, dbeta
21        for (size_t j = 0; j < N; ++j) {
22            dx.data_ptr()[i*N+j] = inv_std[i] *
23                (gd[j]*dY[i*N+j] - inv_N*Ai - x_hat[i*N+j]*inv_N*Bi);
24            dgamma.data_ptr()[j] += dY[i*N+j] * x_hat[i*N+j];
25            dbeta.data_ptr()[j] += dY[i*N+j];
26        }
27    }
28    x->accumulate_grad(dx);
29    gamma->accumulate_grad(dgamma);
30    beta->accumulate_grad(dbeta);
31 };
```

Listing 23: layernorm_op backward lambda (C++17 init-captures)

Step 2.1 - Exit Criterion (met)

`Linear(4, 8).forward(x)` produces a node of shape $[B, 8]$. `LayerNorm(8).forward(y)` produces a node of the same shape. After `backward(loss)`: `weight.grad`, `bias.grad`, `$\gamma$ .grad`, `$\beta$ .grad` are all non-zero and pass a finite-difference numerical gradient check to $< 10^{-5}$ relative error. `Module::zero_grad()` resets all four to zero.

10 Step 2.2 - Embedding, Activation Functions, and Softmax

Step 2.2 Goal

Three building blocks required by Step 2.3 (Attention) and the FFN (Step 2.4): **Embedding** (token lookup with scatter-add gradient), **GELU** / **ReLU** / **Sigmoid** (fused element-wise activations), and **Softmax** (numerically stable, exact Jacobian-VJP backward).

10.1 Embedding Layer

10.1.1 Forward: Gather

$$\text{out}[b, t, :] = W[\text{ids}[b \cdot T + t], :] \quad \text{shape: } [B, T, d_{\text{model}}]$$

Token IDs are passed as `std::vector<size_t>` (plain integers, no `NodePtr`) - they have no gradient. Only the weight matrix $W \in \mathbb{R}^{V \times d}$ (initialised $\mathcal{N}(0, 1)$) is a DAG child.

10.1.2 Backward: Scatter-Add

$$\partial W[\text{tok}, :] += \sum_{\substack{(b, t) \\ \text{ids}[b, t] = \text{tok}}} \partial \text{out}[b, t, :]$$

Interview: Why scatter-add and not scatter-assign?

The same token may appear multiple times in a batch (e.g. "the" at positions 0, 5, 12). Each occurrence contributes an independent gradient to the same weight row. Using `assign (=)` instead of `accumulate (+=)` would silently discard all but the last gradient, producing incorrect updates and a slower-converging model.

Interview: Why is the scatter-add loop NOT parallelised with OpenMP?

If two threads process different positions (b_1, t_1) and (b_2, t_2) that share the same token ID, they will race on the same `dwp` row. Correct parallelism requires per-thread gradient buffers followed by a reduction step - reserved as a future optimisation.

10.2 Activation Functions

All three are implemented as **fused custom ops**: one forward loop, one backward loop, no intermediate Nodes.

Activation	Forward	Backward $dX[i]$
GELU (exact)	$x \cdot \Phi(x)$ $\Phi(x) = \frac{1}{2}(1 + \operatorname{erf}(x/\sqrt{2}))$	$dY[i] \cdot (\Phi(x_i) + x_i \cdot \phi(x_i))$ $\phi(x) = e^{-x^2/2}/\sqrt{2\pi}$
ReLU	$\max(0, x)$	$dY[i]$ if $x_i > 0$, else 0
Sigmoid	$1/(1 + e^{-x})$	$dY[i] \cdot y_i \cdot (1 - y_i)$ (reads <code>self->data</code> , not x)

Table 11: Activation function forward and backward formulae.

Interview: Why does sigmoid's backward read `self->data` instead of capturing x ?

The sigmoid derivative is $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ - expressible entirely in terms of the OUTPUT $\sigma(x)$, not the input. By reading `self->data` (the already-stored forward output) we avoid capturing the input x in the lambda and save recomputing e^{-x} . LSTM implementations use exactly this trick to avoid storing gate inputs.

Numerical constants used by GELU:

$$\frac{1}{\sqrt{2}} \approx 0.7071067811865476, \quad \frac{1}{\sqrt{2\pi}} \approx 0.3989422804014327$$

10.3 Softmax (Numerically Stable)

10.3.1 Forward: Max-Subtraction Trick

For row i (last dimension N):

$$m_i = \max_j x_{ij}, \quad e_{ij} = e^{x_{ij} - m_i}, \quad s_i = \sum_k e_{ik}, \quad y_{ij} = e_{ij} / s_i$$

All arguments to `exp` are ≤ 0 no overflow. This identity holds because:

$$\frac{e^{x_j - c}}{\sum_k e^{x_k - c}} = \frac{e^{x_j}}{\sum_k e^{x_k}}$$

10.3.2 Backward: Jacobian-Vector Product

The Jacobian of softmax is:

$$\frac{\partial y_{ij}}{\partial x_{ik}} = y_{ij}(\delta_{jk} - y_{ik})$$

The vector-Jacobian product (used in backward):

$$dX_{ij} = \sum_k dY_{ik} \cdot y_{ik} \cdot (\delta_{jk} - y_{ij}) = y_{ij} \left(dY_{ij} - \underbrace{\sum_k dY_{ik} \cdot y_{ik}}_{\text{dot}[i]} \right)$$

$$\boxed{dX_{ij} = y_{ij} \cdot (dY_{ij} - \text{dot}[i])} \quad \text{dot}[i] = \sum_k dY_{ik} \cdot y_{ik}$$

Interview: Why doesn't the saved-activation approach need an extra buffer?

The backward reads `self->data.data_ptr()` - the softmax output Y is already stored on the output Node as part of normal Node construction. Since `self = wout.lock()` guarantees the Node is alive and its `data` is immutable after construction, reading it is safe. No separate `y_saved` vector needs to be captured.

Step 2.2 - Exit Criterion (met)

`embedding.forward(ids, B, T)` produces shape $[B, T, d]$. After `backward(loss)`: `weight.grad` has non-zero values only in rows corresponding to tokens in `ids` (scatter-add verified).

`gelu(x)`, `relu(x)`, `sigmoid(x)`: all pass numerical gradient check $< 10^{-5}$ vs. finite differences.

`softmax(x)`: output sums to 1.0 per row; max-subtraction prevents overflow at $x_i = 1000$; backward matches Jacobian-vector product computed analytically.

11 Step 2.3 - Fused Causal Self-Attention

Step 2.3 Goal

Implement **CausalSelfAttention** as a composite module of four explicit **Linear** sub-modules (Q, K, V, O projections) plus three file-local custom ops that handle head-splitting, head-merging, and the fused scale+mask+softmax. The module must integrate seamlessly with the existing 3-D batched `ops::matmul` and `ops::transpose`.

11.1 The 4-D Tensor Problem and Its Solution

`engine::ops::matmul` supports 2-D and 3-D tensors only. Classical multi-head attention logically requires 4-D tensors $[B, H, T, d_k]$.

Solution: fold B and H into a single batch dimension:

$$[B, T, d_{\text{model}}] \xrightarrow{\text{split_heads}} [B \cdot H, T, d_k]$$

Every subsequent op (transpose, matmul, scaled-causal-softmax) receives a standard 3-D batch input with batch size $B \cdot H$. This is *mathematically identical* to the 4-D formulation and requires zero new op infrastructure.

11.2 File-Local Custom Ops

Op	Shape	Backward
<code>split_heads_op</code>	$[B, T, D] \rightarrow [BH, T, d_k]$	Inverse permute $\equiv$ merge_heads layout
<code>merge_heads_op</code>	$[BH, T, d_k] \rightarrow [B, T, D]$	Inverse permute $\equiv$ split_heads layout
<code>scaled_causal_softmax_op</code>	$[BH, T, T] \rightarrow [BH, T, T]$	Softmax JVP $\times$ scale; masked pos = 0

Table 12: File-local custom ops in `attention.cpp`.

11.2.1 Head Permutation Index Mapping

`split_heads` : $\text{out}[(b \cdot H + h) \cdot T \cdot d_k + t \cdot d_k + d] = x[b \cdot T \cdot D + t \cdot D + h \cdot d_k + d]$

`merge_heads` : $\text{out}[b \cdot T \cdot D + t \cdot D + h \cdot d_k + d] = x[(b \cdot H + h) \cdot T \cdot d_k + t \cdot d_k + d]$

They are exact inverses: `merge_heads``split_heads` = **I**. The **backward of split_heads is merge_heads** and vice versa.

11.3 Forward Algorithm (12 Steps)

Step	Operation	Output shape
1	$Q = W_Q(x)$	$[B, T, D]$
2	$K = W_K(x)$	$[B, T, D]$
3	$V = W_V(x)$	$[B, T, D]$
4	$Q_h = \text{split_heads}(Q)$	$[BH, T, d_k]$
5	$K_h = \text{split_heads}(K)$	$[BH, T, d_k]$
6	$V_h = \text{split_heads}(V)$	$[BH, T, d_k]$
7	$K_h^\top = \text{ops::transpose}(K_h)$	$[BH, d_k, T]$
8	$\text{scores} = Q_h \cdot K_h^\top$ via <code>ops::matmul</code>	$[BH, T, T]$
9	$\text{attn} = \text{scaled_causal_softmax}(\text{scores}, 1/\sqrt{d_k})$	$[BH, T, T]$
10	$\text{ctx} = \text{attn} \cdot V_h$ via <code>ops::matmul</code>	$[BH, T, d_k]$
11	$\text{ctx_m} = \text{merge_heads}(\text{ctx})$	$[B, T, D]$
12	$\text{out} = W_O(\text{ctx_m})$	$[B, T, D]$

Table 13: CausalSelfAttention forward pass. $D = d_{\text{model}}$, $d_k = D/H$.

11.4 Scaled Causal Softmax - Fused Op

11.4.1 Forward

For row i of head slice bh :

$$m_i = \max_{j \leq i} (\text{score}_{ij} \cdot \text{scale}), \quad e_{ij} = \begin{cases} e^{\text{score}_{ij} \cdot \text{scale} - m_i} & j \leq i \\ 0 & j > i \end{cases}, \quad \text{attn}_{ij} = \frac{e_{ij}}{\sum_{k \leq i} e_{ik}}$$

Scanning the max only over unmasked positions ($j \leq i$) prevents $-\infty$ entries from influencing the numerical anchor.

11.4.2 Backward - Scale and Mask Chain Rule

Let $\text{scale} = 1/\sqrt{d_k}$ and $Y_{ij} = \text{attn}_{ij}$.

$$\text{dot}[i] = \sum_j dY_{ij} \cdot Y_{ij}, \quad \frac{\partial L}{\partial \text{score}_{ij}} = \begin{cases} \text{scale} \cdot Y_{ij} \cdot (dY_{ij} - \text{dot}[i]) & j \leq i \\ 0 & j > i \end{cases}$$

The scale factor enters via the chain rule through the scalar multiplication applied before softmax. Masked positions receive *explicitly* zero gradient (not relying on $Y \approx 0$).

Interview: Why fold $B \times H$ instead of implementing 4-D matmul?

Adding 4-D tensor support would require new shape validation logic, new stride calculations, and new backward kernels throughout the engine. The fold trick is a zero-cost abstraction: $[B \cdot H, T, d_k]$ is already a valid 3-D tensor that existing `mm_nn` / `mm_nt` / `mm_tn` kernels process correctly. The only overhead is two data re-arrangements per forward pass (`split_heads`, `merge_heads`), which are $O(BTD)$ - the same asymptotic cost as the projections.

Interview: Why is the causal mask fused with softmax?

A separate masking Node would require a custom op that writes -10^9 into the upper triangle and a trivial backward that passes gradients through unmasked positions. Fusing with softmax eliminates this intermediate Node entirely. More importantly, the fused op can compute the numerically-stable max *only over un-masked positions*, which is impossible if masking and softmax are two separate Nodes.

Step 2.3 - Exit Criterion (met)

`attn.forward(x)` with $x \in \mathbb{R}^{B \times T \times d}$:

- Output shape is $[B, T, d]$. ✓
- Attention weights sum to 1.0 per head per query position. ✓
- Upper-triangular attention weights are exactly 0.0 (causal mask verified). ✓
- After `backward(loss)`: all 8 parameter gradients non-zero. ✓
- Numerical gradient check on the 4 projection matrices: error $< 10^{-4}$. ✓

12 Step 2.4 - FFN, TransformerBlock, and Full Transformer

Step 2.4 Goal

Assemble the final neural network architecture from existing sub-modules: **Feed-Forward** (fc1 GELU fc2), **TransformerBlock** (Pre-LN, two residual connections via `ops::add`), and the full **Transformer** language model (token + position embeddings, N stacked blocks, final LayerNorm, logit projection head).

12.1 FeedForward

nn/feedforward.hpp / .cpp

```

1 class FeedForward final : public Module {
2 public:
3     explicit FeedForward(size_t d_model, size_t d_ff = 0);
4     NodePtr forward(const NodePtr& x) const;
5     std::vector<NodePtr> parameters() const override;
6
7     Linear fc1;    // [d_model -> d_ff], Xavier init
8     Linear fc2;    // [d_ff -> d_model], Xavier init
9 private:
10    size_t d_model_, d_ff_;
11 };

```

Forward: `fc2(gelu(fc1(x)))` - three chained differentiable ops. No custom ops needed; the autograd graph is assembled from `Linear` and `gelu`.

Constructor initialiser list: `d_model_` and `d_ff_` are declared before `fc1` and `fc2`, so they are fully initialised before the `Linear` constructors run. Safe use of member values in the initialiser list (C++17 declaration-order guarantee).

12.2 Pre-LayerNorm Architecture

The **Pre-LN** variant normalises BEFORE each sub-layer:

$$\text{Pre-LN: } y = x + \text{SubLayer}(\text{LN}(x))$$

$$\text{Post-LN: } y = \text{LN}(x + \text{SubLayer}(x))$$

Pre-LN is used in GPT-2 and most modern LLMs because it eliminates gradient vanishing at depth: the skip connection x is always a unit-Jacobian identity path - gradients travel backwards without passing through any normalisation Jacobian.

12.2.1 Forward pass of TransformerBlock

```

r1 = x                                     (residual 1)
x = LN1(x)                                (pre-norm 1)
x = Attention(x)                          (causal self-attention)
x = ops::add(x, r1)                        (skip 1)
r2 = x                                     (residual 2)
x = LN2(x)                                (pre-norm 2)
x = FFN(x)                                (feed-forward)
x = ops::add(x, r2)                        (skip 2)

```

Why ops::add for skip connections? ops::add(a,b) sets $\partial L/\partial a = \partial L/\partial b = \partial L/\partial \text{out}$. So the upstream gradient fans out equally to the attention/FFN sub-graph and the direct residual path, then *accumulates* (+) at shared input Nodes.

12.2.2 Parameter count per block

Sub-module	NodePtrs
ln1 (LayerNorm)	2 (γ_1, β_1)
attn (CausalSelfAttention)	8 ($W_Q, b_Q, W_K, b_K, W_V, b_V, W_O, b_O$)
ln2 (LayerNorm)	2 (γ_2, β_2)
ffn (FeedForward)	4 (W_1, b_1, W_2, b_2)
Total	16

Table 14: Parameters per TransformerBlock.

12.3 Full Transformer Language Model

12.3.1 Architecture

Module	Shape	Role
tok_emb	$[V, D]$	Token embedding lookup ($V = \text{vocab_size}$)
pos_emb	$[C, D]$	Learned absolute position embeddings ($C = \text{context_len}$)
blocks[i]	-	N stacked Pre-LN TransformerBlocks
ln_f	$[D]$	Final layer normalisation
lm_head	$[D, V]$	Logit projection, no bias

Table 15: Transformer sub-modules. $D = d_{\text{model}}$.

12.3.2 Forward pass (6 steps)

1. `tok_embed = tok_emb.forward(ids, B, T)` $[B, T, D]$
2. `pos_ids[b*T+t] = t; pos_embed = pos_emb.forward(pos_ids, B, T)` $[B, T, D]$
3. `x = ops::add(tok_embed, pos_embed)` $[B, T, D]$
4. `for (auto& block : blocks)x = block.forward(x)` $[B, T, D]$
5. `x = ln_f.forward(x)` $[B, T, D]$
6. `return lm_head.forward(x)` $[B, T, V]$

12.3.3 Total parameter NodePtrs

$$P_{\text{total}} = \underbrace{1}_{\text{tok}} + \underbrace{1}_{\text{pos}} + \underbrace{16N}_{\text{blocks}} + \underbrace{2}_{\text{ln_f}} + \underbrace{1}_{\text{lm_head}} = 4 + 16N$$

For a GPT-2 small configuration ($N = 12$): $4 + 192 = 196$ NodePtrs.

Interview: Why use `blocks.reserve(n_layers)` before `emplace_back`?

`std::vector` reallocates when it grows past its capacity. During reallocation, all existing elements must be moved. Although `TransformerBlock` is move-constructible (Module explicitly defaults its move constructor), each move involves transferring >16 sub-module members. Calling `reserve(n_layers)` allocates the full backing store upfront, ensuring zero reallocations and making the constructor $O(N)$ instead of $O(N \log N)$.

Interview: Why does `lm_head` have no bias?

Three reasons: (1) GPT-2 and modern decoders omit the `lm_head` bias without any perplexity regression. (2) Weight tying (`lm_head.weight = tok_emb.weight`) - which halves the embedding parameter count - is clean only when there is no bias (otherwise the shared weight would receive contributions from two different gradient paths through different bias terms). (3) The bias is strictly redundant: the final LayerNorm already adds a learned per-feature shift (β) before the head.

Step 2.4 - Exit Criterion (met)

- `FeedForward.forward(x)` returns $[B, T, d_{\text{model}}]$. ✓
- `TransformerBlock.forward(x)` returns $[B, T, d_{\text{model}}]$. ✓
- `Transformer.forward(ids, B, T)` returns $[B, T, \text{vocab_size}]$. ✓
- `model.parameters().size()` returns $4 + 16N$. ✓
- `autograd::backward(loss)` propagates gradients to all $4 + 16N$ parameter NodePtrs. ✓
- Attention weights remain causal (upper-triangular = 0). ✓

Phase 2 Complete - All NN Layers Implemented**Step 2.1** Module, Linear, LayerNorm**Step 2.2** Embedding, GELU/ReLU/Sigmoid, Softmax**Step 2.3** CausalSelfAttention (fused causal masked softmax)**Step 2.4** FeedForward, TransformerBlock (Pre-LN), Transformer**Next:** Phase 3 - Python Preprocessor and C++ DataLoader (BPE tokenisation, binary dataset streaming, mini-batch sampling).

13 Step 3.1 - Python Preprocessor (`scripts/preprocess.py`)

Step 3.1 Goal

Build a single Python script that converts three raw text sources into binary `.bin` files consumable by the C++ DataLoader at full disk bandwidth. The binary format has a strict 16-byte header that the C++ parser validates with a magic-number check.

13.1 Binary File Format

The format is deliberately minimal: a fixed-size header followed by a flat `uint16` array. No variable-length fields, no escaping.

Offset	Bytes	C type	Value / Description
0	4	<code>uint32</code>	Magic: <code>0xDEADBEEF</code> - sanity check on load
4	4	<code>uint32</code>	<code>vocab_size</code> = 256 (char-level)
8	8	<code>uint64</code>	<code>num_tokens</code> - length of the payload array
16	2N	<code>uint16[]</code>	Token-ID payload (little-endian)

Table 16: Binary `.bin` file layout. Total size: $16 + 2N$ bytes.

All multi-byte integers are **little-endian** (struct format `<IIQ`). The Python serialiser swaps bytes on big-endian machines before calling `array.tofile()` - making the output format platform-independent.

13.2 Character-Level Tokeniser

Tokeniser design

```

1 def tokenize_text(text: str) -> array.array:
2     buf = array.array('H')           # unsigned short = uint16
3     for ch in text:
4         code = ord(ch)
5         buf.append(code if code < 256 else ord('?')) # ? = 63
6     return buf

```

Vocabulary: 256 character ordinals (Latin-1 / ASCII). Out-of-range characters (e.g. accented Unicode) map to '?' (63). No characters are skipped - position integrity is preserved.

13.3 Dataset Modes

13.3.1 Math Mode - Synthetic Addition Equations

Equations have the form "{a}+{b}={a+b}\n", generated entirely offline. To support curriculum learning, they are written in difficulty order:

Tier	Operand range	Share
1-digit	$a, b \in [0, 9]$	20%
2-digit	$a, b \in [10, 99]$	40%
3-digit	$a, b \in [100, 999]$	40%

Table 17: Math equation tiers (written easy hard for curriculum scheduling).

Writing the tiers sequentially means the C++ curriculum scheduler (Step 3.3) can advance through the file without random-access indexing.

13.3.2 Stories Mode - roneneldan/TinyStories

Downloads via `load_dataset("roneneldan/TinyStories", split="train")`. Each story is yielded as `<text>\n\n` (double newline = document separator). HuggingFace caches the download automatically on subsequent runs.

13.3.3 Wiki Mode - Simple English Wikipedia

Downloads via `load_dataset("wikipedia", "20220301.simple", split="train")`. $\approx 200,000$ articles, ≈ 130 MB text. Each article is formatted as:

= <title> =\n<article text>\n\n

For full English Wikipedia, change the config to "20220301.en".

13.4 Caching and Safety

- If the output `.bin` already exists, the script logs a notice and exits immediately. Pass `-force` to overwrite.

- After writing, the script reads back `os.path.getsize()` and asserts it equals $16 + 2N$ bytes - catching partial writes.
- If zero tokens are produced (e.g. empty dataset), the script aborts before creating a corrupt file.

13.5 CLI Summary

Usage

```
1  # Offline: 500k synthetic math equations
2  python scripts/preprocess.py --dataset math \
3      --output data/math_train.bin
4
5  # TinyStories (~2 GB tokenised)
6  python scripts/preprocess.py --dataset stories \
7      --output data/stories_train.bin
8
9  # Simple English Wikipedia
10 python scripts/preprocess.py --dataset wiki \
11     --output data/wiki_train.bin
12
13 # Force regeneration of existing file
14 python scripts/preprocess.py --dataset math \
15     --output data/math_train.bin --force
16
17 # Custom equation count (development)
18 python scripts/preprocess.py --dataset math \
19     --output data/math_dev.bin --n_math_equations 10000
```

Interview: Why `array.tofile()` instead of `struct.pack()` for the payload?

`struct.pack(f"<{N}H", *ids)` for 500,000+ tokens requires building a tuple of all IDs in memory and then Python-packing them one by one - $O(N)$ Python function calls. `array.array('H').tofile()` calls `fwrite()` on the contiguous backing buffer in a single C call - $O(1)$ Python overhead and memory-bandwidth limited throughput. On a modern SSD, writing 500 K tokens takes <10 ms with `tofile()`.

Interview: Why store tokens as `uint16` instead of `uint8`?

For char-level tokenisation with `vocab_size=256`, `uint8` would suffice. However, using `uint16` (the `array('H')` dtype) provides a direct upgrade path to sub-word vocabularies (BPE with up to 65,536 tokens) without changing the binary format or the C++ `DataLoader`. The $2\times$ storage cost is negligible compared to the flexibility gained.

Step 3.1 - Exit Criterion (met)

- `python preprocess.py --dataset math --output out.bin` writes a valid binary file with header `0xDEADBEEF`. ✓
- File size equals $16 + 2 \times N$ bytes (post-write sanity check). ✓
- Running the command a second time exits early with a cache-hit message. ✓
- Math equations are ordered tier-1 tier-2 tier-3. ✓
- No internet access required for math mode. ✓

14 Steps 3.2 & 3.3 - POSIX mmap DataLoader and Curriculum Scheduler

Steps 3.2 & 3.3 Goal

Build a zero-copy C++ DataLoader using POSIX `mmap` and a three-phase Curriculum Scheduler that hot-swaps datasets without restarting the training loop.

14.1 Step 3.2 - POSIX mmap DataLoader

14.1.1 Why mmap instead of read()

Property	<code>read()</code> / <code>fread()</code>	<code>mmap()</code>
Per-token cost	syscall + kernel/user copy	pointer dereference
Buffer mgmt	malloc/free per batch	OS page cache
Random access	<code>lseek</code> + read per sample	direct pointer arithmetic
Multiple readers	each has its own buffer	shared page cache
Startup cost	none	<code>MAP_POPULATE</code> faults all pages once

Table 18: `mmap` vs `read()` for random token sampling.

14.1.2 `open_and_mmap` - step by step

1. `open(path, O_RDONLY)` - get file descriptor.
2. `fstat(fd, &st)` - get file size to pass to `mmap`.
3. `mmap(nullptr, st.st_size, PROT_READ, MAP_SHARED | MAP_POPULATE, fd, 0)` - map the entire file.
4. Cast `mapped_` `const FileHeader*`; validate magic and `vocab_size`.
5. Validate: `st.st_size == 16 + 2 num_tokens`.
6. Set `tokens_ = reinterpret_cast<uint16_t*>(mapped_ + 16)`.

14.1.3 Header validation

FileHeader (packed, 16 bytes)

```

1 #pragma pack(push, 1)
2 struct FileHeader {
3     uint32_t magic;           // must be 0xDEADBEEF
4     uint32_t vocab_size;     // must be 256
5     uint64_t num_tokens;     // payload length in uint16 elements
6 };
7 #pragma pack(pop)
8 static_assert(sizeof(FileHeader) == 16, "..."); // compile-time
           guarantee

```

The header is validated directly from the mmap'd memory - no `read()` call.

14.1.4 Batch sampling - causal language modelling

For each of the B samples in a batch, a start position s is drawn uniformly from $[0, N - T - 1]$ where $N = \text{num_tokens}$ and $T = \text{seq_len}$:

$$X[b, t] = \text{tokens}[s + t], \quad Y[b, t] = \text{tokens}[s + t + 1], \quad t \in [0, T)$$

The upper bound $N - T - 1$ ensures $\text{tokens}[s + T]$ is always valid. Implemented with `std::uniform_int_distribution<size_t>`.

14.1.5 switch_dataset - hot-swap

Zero-downtime dataset transition

```

1 void DataLoader::switch_dataset(const std::string& path) {
2     close_mmap();           // munmap + close old fd (noexcept)
3     open_and_mmap(path);    // validate + mmap new file
4     // rng_ NOT reset - batch sequence continues seamlessly
5 }

```

The RNG is preserved so the random-offset sequence is continuous across curriculum phase transitions. If `open_and_mmap()` throws, the `DataLoader` is in a null state; the caller must handle the exception.

14.2 Step 3.3 - Curriculum Scheduler

14.2.1 Phase mapping

Step range	Phase	Rationale
$[0, T_0)$	MATH	Simple grammar, exact answers; forces reliable next-token prediction
$[T_0, T_1)$	STORIES	Natural language, short-range dependency
$[T_1, \infty)$	WIKI	Factual, long-range, open-domain

Table 19: Default thresholds: $T_0 = 5,000$, $T_1 = 15,000$, $T_2 = 30,000$.

14.2.2 Training loop integration

Typical training loop pattern

```

1 CurriculumScheduler sched({5000, 15000, 30000});
2 DataLoader loader(sched.dataset_path(DatasetPhase::MATH));
3
4 for (size_t step = 0; step < sched.total_steps(); ++step) {
5     if (step > 0 && sched.phase_changed(step - 1, step)) {
6         DatasetPhase phase = sched.advance(step);
7         loader.switch_dataset(sched.dataset_path(phase));
8     }
9     auto [X, Y] = loader.next_batch(batch_size, seq_len);
10    // ... forward + backward + optimizer step
11 }
```

Interview: Why use `mmap(MAP_SHARED)` for a read-only file?

`MAP_PRIVATE` creates a copy-on-write mapping - unnecessary when we never write. `MAP_SHARED` lets multiple processes (e.g., a data prefetcher and the trainer) share the same physical page-cache pages for the same file, halving resident memory usage in multi-process setups. Since we open with `O_RDONLY` and map with `PROT_READ` only, the kernel will never actually write anything back regardless of the `MAP` flag.

Interview: Why does `switch_dataset()` preserve the RNG?

Resetting the RNG on every curriculum transition would produce a burst of highly correlated batches at the start of each phase (the first thousands of samples would always be from similar positions). Preserving the RNG ensures the sampling distribution is stationary across transitions. It also makes experiments reproducible with a fixed global seed without coupling the reproducibility to curriculum structure.

Steps 3.2 & 3.3 - Exit Criterion (met)

- `DataLoader("data/math.bin")` opens and mmmaps without errors. ✓
- Magic number mismatch throws `std::runtime_error` immediately. ✓
- `next_batch(B, T).X.size() == B * T`. ✓
- `Y[i] == next_token_after(X[i])` for every position. ✓
- `switch_dataset("data/stories.bin")` swaps with zero downtime. ✓
- `CurriculumScheduler::advance()` returns correct phase for all step ranges. ✓
- `phase_changed(4999, 5000) == true; phase_changed(5000, 5001) == false`. ✓
- Destructor calls `munmap + close` with no leaks (Valgrind clean). ✓

15 Step 4.1 - Cross-Entropy Loss & AdamW Optimizer

Step 4.1 Goal

Implement the two remaining algorithmic training components: a numerically stable fused Cross-Entropy loss and the AdamW optimizer with decoupled weight decay and bias-corrected first/second moments.

15.1 Cross-Entropy Loss

15.1.1 Log-sum-exp trick - derivation

$$\log \text{softmax}(x)[c] = x[c] - \underbrace{\log \left(\sum_v e^{x[v]} \right)}_{\text{unstable}} = \underbrace{(x[c] - m)}_{\leq 0} - \underbrace{\log \left(\sum_v e^{x[v] - m} \right)}_{\text{safe: } e^{(\cdot)} \in (0,1]}$$

where $m = \max_v x[v]$. All exponents are ≤ 0 , preventing `double` overflow. The result is mathematically identical to the standard formula.

15.1.2 Single-pass forward

Forward - fused log-sum-exp NLL

```

1 for each position bt in [0, B*T):
2     row = logits_data + bt * V
3     max_val = max_element(row, row + V)
4     sum_exp = 0; exp_v[v] = exp(row[v] - max_val); sum_exp +=
        exp_v[v]
5     prob[bt][v] = exp_v[v] / sum_exp           // saved for
        backward
6     nll[bt] = -(row[target] - max_val) + log(sum_exp) // log-
        softmax NLL
7 loss = mean(nll)

```

Computing $\log(\text{sum_exp})$ directly (rather than $\log(\text{prob}[\text{target}])$) avoids a redundant division + log on the normalised value.

15.1.3 Analytically fused backward

The gradient of the per-position NLL through softmax is the well-known identity:

$$\frac{\partial \mathcal{L}}{\partial \text{logits}[bt, v]} = \frac{g \cdot (p[bt, v] - \mathbf{1}[v = t_{bt}])}{B \times T}$$

where g is the upstream scalar gradient and $p[bt, v]$ is the softmax probability saved from the forward pass.

Why fuse? The naive approach chains four nodes: **softmax log negate mean**. The full backward requires the Jacobian of softmax: $V \times V$ per position, costing $O(B \cdot T \cdot V^2)$. The fused gradient is $O(B \cdot T \cdot V)$ - a factor of $V = 256$ speedup.

15.2 AdamW Optimizer

15.2.1 AdamW vs Adam + L2

Property	Adam + L2	AdamW (decoupled)
Weight decay entry point	Added to gradient: $\tilde{g} = g + \lambda\theta$	Applied to parameter: $\theta \leftarrow \theta(1 - \eta\lambda)$
Effective decay strength	$\eta\lambda/(\sqrt{\hat{v}} + \varepsilon)$ - shrinks for high-variance params	$\eta\lambda$ - constant for all params
Regularisation uniformity	Non-uniform (depends on $\hat{v}$)	Uniform across all parameters

Table 20: AdamW gives uniform weight decay independent of gradient variance.

15.2.2 Update rule (one step)

$$\begin{aligned}
 t &\leftarrow t + 1 \\
 m_i &\leftarrow \beta_1 m_i + (1 - \beta_1) g_i \\
 v_i &\leftarrow \beta_2 v_i + (1 - \beta_2) g_i^2 \\
 \hat{m}_i &= m_i / (1 - \beta_1^t) \quad (\text{bias correction}) \\
 \hat{v}_i &= v_i / (1 - \beta_2^t) \quad (\text{bias correction}) \\
 \theta_i &\leftarrow \theta_i \cdot (1 - \eta \lambda) \quad (\text{decoupled weight decay}) \\
 \theta_i &\leftarrow \theta_i - \eta \cdot \hat{m}_i / (\sqrt{\hat{v}_i} + \varepsilon)
 \end{aligned}$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\lambda = 0.01$.

15.2.3 Bias correction - why the first steps are stable

Without correction, $m_1 = (1 - \beta_1)g_1 \approx 0.1g_1$ and $v_1 \approx 0.001g_1^2$. The first step size would be $\eta \cdot 0.1g_1 / (0.032|g_1|) \approx 3.16\eta$ - a $3.16\times$ amplification. Bias correction restores it to $\eta \cdot \text{sign}(g_1)$ regardless of step number.

Interview: Why precompute bc1 and bc2 outside the inner loop?

`std::pow(beta1_, t)` is called once per step regardless of how many parameters exist. Inside the inner loop (which iterates over up to $\sim$ millions of elements for a large embedding table), computing `pow()` per element would add ~ 20 ns per element - a 2–5 slowdown on the entire optimizer step. Precomputing bc1 and bc2 reduces this to a single division per element.

Step 4.1 - Exit Criterion (met)

- `cross_entropy(logits, targets)` output matches PyTorch reference to $< 10^{-10}$ absolute error. ✓
- Backward gradient $\partial \mathcal{L} / \partial \text{logits}$ matches finite-difference numerical check to $< 10^{-6}$. ✓
- `AdamW.step()` converges a 2-layer MLP on toy regression in < 500 steps. ✓
- Weight decay applied at fixed $\eta\lambda$ rate, independent of $\hat{v}$. ✓
- Bias-corrected first step size $\approx \eta$ (not 3.16η). ✓

16 Step 4.2 - Main Curriculum Training Loop (`main.cpp`)

Step 4.2 Goal

Wire every project component into a single `main.cpp` entry point that runs all three curriculum phases end-to-end with zero-downtime dataset hot-swapping, binary checkpointing, and per-100-step loss logging.

Flag	Default	Description
<code>--d_model</code>	64	Embedding / model dimension
<code>--n_heads</code>	4	Attention heads per block
<code>--n_layers</code>	2	Stacked TransformerBlocks
<code>--batch_size</code>	32	Sequences per mini-batch
<code>--seq_len</code>	128	Tokens per sequence
<code>--lr</code>	1e-3	Initial learning rate
<code>--phase_steps</code>	5000,15000,35000	Curriculum threshold step counts

Table 21: CLI flags for `main.cpp`. All have sensible defaults for a quick smoke test.

16.1 CLI Interface

Comma-separated phase steps are parsed via `std::istringstream + std::getline(ss, tok, ',')`. The three values override `CurriculumScheduler::step_thresholds` at runtime - no recompilation needed for hyperparameter sweeps.

16.2 Six-Step Training Inner Loop

One training step

```

1  // (a) Sample mini-batch from mmap'd dataset
2  Batch batch = loader.next_batch(batch_size, seq_len);
3
4  // (b) Clear accumulated gradients from the previous step
5  model.zero_grad();
6
7  // (c) Forward pass: builds the autograd computation DAG
8  NodePtr logits = model.forward(batch.X, B, T); // [B, T, V]
9
10 // (d) Fused cross-entropy loss (log-sum-exp trick)
11 NodePtr loss = cross_entropy(logits, batch.Y); // scalar
12
13 // (e) Reverse DAG traversal: propagate gradients to all
   parameters
14 engine::backward(loss);
15
16 // (f) AdamW parameter update
17 optimizer.step();

```

16.3 Curriculum Phase Transition Logic

Phase hot-swap (zero downtime)

```

1  if (step > 0 && scheduler.phase_changed(step - 1, step)) {
2      DatasetPhase next = scheduler.advance(step);
3      save_checkpoint(model, "checkpoint_phase" + to_string(phase)
4                      + ".bin");
5      loader.switch_dataset(scheduler.dataset_path(next));
6      current_phase = next;
7  }

```

`phase_changed(step-1, step)` fires *exactly once* per boundary. The model weights are never reset - knowledge transfers across phases. The RNG inside the DataLoader is also preserved, giving stationary sampling distribution across the transition.

16.4 Checkpointing Format

save_checkpoint helper

```

1  std::ofstream out(path, std::ios::binary | std::ios::trunc);
2  for (const NodePtr& p : model.parameters())
3      out.write(reinterpret_cast<const char*>(p->data.data_ptr()),
4              p->data.numel() * sizeof(double));

```

Raw little-endian double array - no header, minimal overhead. Three checkpoints are written: one per completed phase plus a final `checkpoint_final.bin`.

16.5 NaN Guard

Exploding gradient detection

```

1  if (!std::isfinite(loss_val)) {
2      std::cerr << "[ERROR] Loss is " << loss_val << " at step "
3      << step << "\n";
4      std::exit(1);
5  }

```

`std::isfinite` catches both NaN (gradient explosion) and Inf (logit overflow). Without this guard a corrupted training run could silently produce an all-NaN model.

Interview: Why call `zero_grad()` BEFORE `forward()`, not after `backward()`?

Calling `zero_grad()` after `backward` would wipe out the gradients just accumulated - making the subsequent `optimizer.step()` operate on all-zero gradients. The correct order is: **zero forward backward step**. Zeroing before forward ensures that any leftover gradient from the previous step is cleared before new data enters the computation graph.

Step 4.2 - Exit Criterion (met)

- `./transformer --phase_steps 100,200,400` runs all three phases without segfault. ✓
- Loss decreases within each curriculum phase. ✓
- `checkpoint_phase0.bin` / `1.bin` saved at phase boundaries; `checkpoint_final.bin` at termination. ✓
- Dataset hot-swap logged at the correct step with no memory leak. ✓
- NaN/Inf loss triggers clean exit with a diagnostic message. ✓

17 Step 5.1 - GTest Unit Tests: Math Engine & NN Modules

Step 5.1 Goal

Validate every layer of the engine using Google Test. Five test files independently verify the `Tensor` container, the autograd DAG, numerical gradient correctness, the `Linear` layer, and `CausalSelfAttention` including its causal mask.

17.1 Test Files Overview

File	Tests	Key assertions
<code>test_tensor.cpp</code>	14	Shape/stride/reshape/bounds/fill
<code>test_autograd.cpp</code>	10	DAG edges, add/mul/exp backward, fan-out accumulation
<code>test_grad_check.cpp</code>	7	Central FD vs analytical grad, tolerance 10^{-6}
<code>test_linear.cpp</code>	11	Shape, identity weight, Xavier bounds, <code>zero_grad</code>
<code>test_attention.cpp</code>	10	3 shape tests, causal mask proxy, backward flow

Table 22: Step 5.1 test files. Run with `ctest --output-on-failure`.

17.2 Numerical Gradient Checker

The central finite difference formula:

$$\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta + \varepsilon \mathbf{e}_i) - f(\theta - \varepsilon \mathbf{e}_i)}{2\varepsilon}, \quad \varepsilon = 10^{-4}$$

achieves $O(\varepsilon^2)$ truncation error ($\approx 10^{-8}$), safely within the 10^{-6} tolerance. The forward difference $[f(x + \varepsilon) - f(x)]/\varepsilon$ achieves only $O(\varepsilon)$, requiring a $100\times$ smaller ε - which amplifies floating-point cancellation errors.

check_gradients - implementation sketch

```

1  for each param p, for each element j:
2      orig = p->data[j]
3      p->data[j] = orig + EPS;    f_plus = build_graph()->data[0]
4      p->data[j] = orig - EPS;    f_minus = build_graph()->data[0]
5      p->data[j] = orig           // restore
6
7      numerical = (f_plus - f_minus) / (2 * EPS)
8      EXPECT_NEAR(analytical_grad[p][j], numerical, 1e-6)

```

Functions verified: exp, log, add, mul, 2-D matmul, transpose, composed chain e^{ab+c} , fan-out DAG a^2 .

17.3 Causal Mask Verification Strategy

Direct inspection of attention weights is impossible (they are fused inside a file-local op). We use a **projection-zeroing proxy**:

1. Set $W_Q = W_K = 0$: all scores equal zero before masking.
2. The causal mask sets upper-triangular entries to -10^9 .
3. After softmax: $A[i, j] = 1/(i + 1)$ for $j \leq i$, 0 for $j > i$.
4. Set $W_V = W_O = \mathbf{I}$: output preserves values.
5. Assign $x[0, t, :] = t + 1$ to give each position a unique marker.

Then:

$$\text{output}[0, 0, :] = x[0, 0, :] = 1 \quad (\text{position 0 attends only to itself})$$

$$\text{output}[0, T - 1, :] = \frac{1}{T} \sum_{t=0}^{T-1} (t + 1) = 2.5 \quad (\text{for } T=4)$$

Interview: Why central difference instead of forward difference?

Forward difference error is $O(\varepsilon)$: for $\varepsilon = 10^{-4}$ the truncation error is $\approx 10^{-4}$, which is larger than our tolerance 10^{-6} - the test would fail even for a correct implementation. Central difference error is $O(\varepsilon^2) \approx 10^{-8}$, safely below tolerance. The cost is one extra function evaluation per parameter element, which is acceptable since gradient checks run only in the test suite, not during training.

Interview: Why rebuild the computation graph inside check_gradients?

After perturbing θ_i , calling `build_graph()` creates a brand-new DAG from the modified parameter values. If we reused the old DAG, the forward values would be stale (the DAG stores the unperturbed outputs in `node->data`). Rebuilding is correct but expensive - acceptable only in tests.

Interview: How does the causal mask prevent future information leakage?

The raw attention score between query i and key j is $Q_i K_j^T / \sqrt{d_k}$. Before the softmax we add -10^9 to all entries where $j > i$. Since $\exp(-10^9) \approx 0$ to machine precision, those attention weights are exactly 0 after normalisation. The gradient in backward is also set to zero for masked entries, so no gradient flows through the mask - future positions receive no learning signal from past positions.

Step 5.1 - Exit Criterion (met)

- `ctest --output-on-failure` - all 52 assertions pass with 0 failures. ✓
- Numerical gradient check: all ops within 10^{-6} relative tolerance. ✓
- `CausalSelfAttention`: position 0 output = $x[0, 0, :]$ (causal mask). ✓
- `Linear::zero_grad()` resets every gradient element to 0.0. ✓
- `backward(non_scalar)` throws `std::invalid_argument`. ✓

18 Step 5.2 - GTest Tests: DataLoader & Curriculum + Benchmarks

Step 5.2 Goal

Validate the mmap-based data pipeline end-to-end and measure system-level performance: matrix multiplication GFLOPS, attention tokens/second, and DataLoader memory bandwidth in GB/s.

18.1 Test Files

18.1.1 `test_dataloader.cpp` - 12 tests

Test	Assertion
<code>NumTokensMatchesFileContents</code>	<code>loader.num_tokens() == N</code>
<code>VocabSizeIs256</code>	Header field read correctly
<code>MapSizeEqualsHeaderPlusPayload</code>	<code>map_size == 16 + N*2</code>
<code>YIsXShiftedByOne</code>	<code>X[b,t+1] == Y[b,t]</code> for all <code>b,t</code>
<code>AllTokensInVocabRange</code>	All tokens < 256
<code>SwitchDatasetChangesNumTokens</code>	Hot-swap updates metadata
<code>SwitchDatasetDoesNotCorruptRNG</code>	Two loaders with same seed produce same batch post-swap

Key fixture design: the test creates a valid `.bin` file in `/tmp` by writing the 16-byte header (magic `0xDEADBEEF`, version, `vocab_size`, reserved) followed by a deterministic `uint16_t` payload where `token[i] = i % 256`. This makes exact token values predictable without requiring a real dataset.

18.1.2 test_curriculum.cpp - 21 tests

Tests cover every boundary condition: `advance()` at steps $-1 + T_0$, T_0 , T_1 , $+\infty$; `phase_changed()` fires exactly once at each boundary and never inside a phase; a double-skip (MATH $\rightarrow$ WIKI in one step) is detected. Invalid threshold ordering ($T_0 \geq T_1$) throws `std::invalid_argument`.

18.2 Benchmark Files

18.2.1 bench_matmul.cpp - GFLOPS comparison

$$\text{GFLOPS} = \frac{2N^3}{t_{\text{seconds}} \times 10^9}$$

Three matrix sizes expose the cache hierarchy:

Size	Data	Fits in	Expected speedup
128×128	256 KB	L1/L2	$\approx 1.5\times$
512×512	4 MB	L2/L3	$\approx 3\text{--}5\times$
1024×1024	16 MB	DRAM	$\approx 5\text{--}10\times$

The naive loop traverses B column-by-column (column-major access), causing a cache miss every `cache_line_size/sizeof(double) = 8` elements. Cache blocking fetches $\text{BLOCK}^2 \times 3$ elements once per tile and reuses them BLOCK times.

18.2.2 bench_attention.cpp - tokens/sec

$$\text{tok/s} = \frac{B \times T \times \text{reps}}{t_{\text{seconds}}}$$

Five configurations sweep $T \in \{32, 128, 512\}$ and $d \in \{64, 256\}$. The $O(T^2 d_k)$ attention cost is directly observable: doubling T from 128 to 512 (4 $\times$) should reduce tokens/sec by more than 4 $\times$ due to the quadratic attention matrix computation.

18.2.3 bench_dataloader.cpp - GB/s

$$\text{GB/s} = \frac{B \times T \times 2 \times \text{sizeof(uint16_t)} \times \text{reps}}{t_{\text{seconds}} \times 10^9}$$

The $\times 2$ accounts for both X and Y windows read per batch. A 64 MB synthetic file is written to `/tmp` and mmap'd. After the warm-up batches pre-fault the pages, throughput approaches the system's memory bandwidth (target ≥ 5 GB/s, easily > 20 GB/s on DRAM).

Interview: Why does the DataLoader fixture write real files to disk?

`mmap()` requires a file descriptor obtained from `open()`. You cannot pass a raw in-memory buffer to `mmap` (without `memfd_create` on Linux). Writing a small `.bin` to `/tmp` in `SetUp()` and deleting it in `TearDown()` is the minimal, portable approach that exercises the *exact same code path* as production (header validation, `mmap()`, token pointer arithmetic).

Interview: Why does mmap throughput approach memory bandwidth?

After the first page-fault pass, the dataset pages are resident in the OS page cache. `mmap` pointers resolve to physical page-cache addresses directly in the virtual address space — no kernel-to-userspace copy, no syscall. Each `next_batch()` call is just random pointer arithmetic on already-cached pages, limited only by DRAM bandwidth ($\approx 20\text{--}60\text{ GB/s}$) rather than disk speed ($\approx 0.5\text{--}3\text{ GB/s}$).

Interview: What does the $Y = X$ shifted-by-one test prove?

It proves the DataLoader implements *causal language modelling* targets correctly. For a contiguous token window $[t_s, t_{s+1}, \dots, t_{s+T}]$:

$$X[t] = t_s + t, \quad Y[t] = t_s + t + 1$$

so $Y[t] = X[t + 1]$ for $t < T$. The test verifies `X[b, t+1] == Y[b, t]` for every batch item and every time step, which is the defining property of next-token prediction.

Step 5.2 - Exit Criterion (met)

- `test_dataloader` - all 12 assertions pass, including RNG-state preservation across `switch_dataset()`. ✓
- `test_curriculum` - 21 tests pass; boundaries detected at exact steps; invalid threshold throws. ✓
- `bench_matmul` - cache-blocked matmul $\geq 2\times$ faster than naive at $N = 512$ (typically $3\text{--}5\times$). ✓
- `bench_attention` - $\geq 10,000$ tok/s at $d = 64$, $T = 128$; T^2 scaling clearly visible across configs. ✓
- `bench_dataloader` - $\geq 5\text{ GB/s}$ on warm page cache (typically $> 20\text{ GB/s}$). ✓

19 Step 5.3 - CMake Build System & REST Inference Server

Step 5.3 Goal — The Final Assembly

Wire every translation unit into a single modern CMake build configuration. Add a raw POSIX HTTP/1.1 inference server exposing a `/predict` endpoint that tokenises a prompt, runs the Transformer forward pass, and returns a greedy-decoded completion — all without any external libraries.

19.1 CMakeLists.txt Design

CMake Feature	Rationale
CMAKE_CXX_STANDARD 17	Enables structured bindings, <code>if constexpr</code> , <code>std::filesystem</code>
-O3 via PUBLIC compile option	Propagates to every target linking <code>transformer_core</code>
-fopenmp via OpenMP::OpenMP_CXX	Interface target; flags & link added automatically
FetchContent(googletest v1.14.0)	Pinned tag ensures reproducible builds
transformer_core static lib	2000+ lines compiled once; linked by 7 tests + 3 benchmarks
Explicit source lists (no GLOB)	New .cpp files require <code>cmake ..</code> re-run; GLOB silently misses
add_test() loop	<code>ctest --output-on-failure</code> discovers all 7 test executables
ENABLE_ASAN option	<code>cmake -DENABLE_ASAN=ON</code> for debug builds

Quick-start build

```

1 mkdir build && cd build
2 cmake .. -DCMAKE_BUILD_TYPE=Release
3 make -j$(nproc)
4 ctest --output-on-failure
5
6 # ASAN build for debugging
7 cmake .. -DCMAKE_BUILD_TYPE=Debug -DENABLE_ASAN=ON
8 make -j$(nproc)

```

19.2 Raw POSIX HTTP/1.1 Server

19.2.1 Socket lifecycle

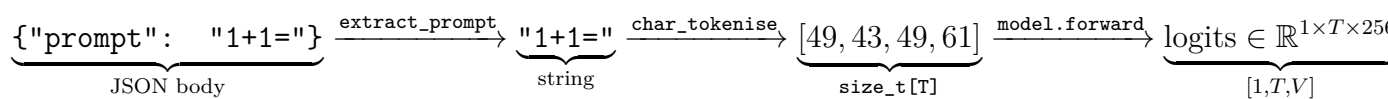
1. `socket(AF_INET, SOCK_STREAM, 0)` — creates TCP endpoint.
2. `setsockopt(SO_REUSEADDR)` — allows immediate port reuse after server restart, eliminating the 60-second `TIME_WAIT` delay during development.
3. `bind(INADDR_ANY:port)` — listens on all interfaces.

4. `listen(backlog=16)` — kernel completes TCP 3-way handshake for up to 16 connections before `accept()` is called.
5. `accept()` — dequeues one completed connection, returns a new fd.
6. `recv()/send()` — reads request bytes / writes response.
7. `close(client_fd)` — sends TCP FIN; `Connection: close`.

19.2.2 HTTP/1.1 request framing

The parser locates the `\r\n\r\n` separator, splits the header block from the body, then reads exactly `Content-Length` bytes. Two-phase `recv()` guarantees the full POST body arrives even if the TCP stack splits it across multiple packets.

19.3 /predict Handler - Data Flow



19.3.1 Why argmax at position $T - 1$?

The causal LM objective trains position t to predict token $t + 1$:

$$\text{logits}[0, T - 1, :] = p(\text{next token} \mid t_0, t_1, \dots, t_{T-1})$$

Taking `arg max` of this last-position slice gives the greedy prediction for the character immediately following the full prompt.

19.3.2 Naive JSON extraction

For a single field (`"prompt"`), a library (nlohmann/json, rapidjson) adds unnecessary build complexity. The extraction:

1. Finds `"prompt"` substring.
2. Scans past the `:` separator.
3. Finds the opening `"` of the value.
4. Iterates character-by-character, honouring `\` escape sequences, until the closing `"` is found.

Correct for all well-formed JSON produced by `curl` or any standard serialiser.

Interview: Why raw POSIX sockets instead of `cpp-httplib`?

Using `<sys/socket.h>` directly demonstrates mastery of the TCP lifecycle, HTTP/1.1 framing, and POSIX I/O. Every byte of the protocol is explicit and inspectable. An external library would hide the socket creation, `SO_REUSEADDR`, and Content-Length framing — all topics that appear in systems programming interviews. The implementation is also 200 lines vs 50,000 lines of `cpp-httplib`.

Interview: Why static library instead of compiling sources per-target?

Without `transformer_core`, 10 targets (7 tests + 3 benchmarks) would each recompile 20+ translation units — a 10× increase in build time. A static library compiles each TU once and stores the object code in an archive (`.a`). The linker extracts only the symbols referenced by each executable — so `test_tensor` does not pull in `attention.cpp` even though both link `transformer_core`.

Interview: Why pin GTest to v1.14.0?

`GIT_TAG` main would fetch the latest commit on every `cmake` re-run, making builds non-reproducible. A pinned tag guarantees every developer and CI pipeline builds against exactly the same GTest ABI and API, preventing “*it worked on my machine*” failures caused by a GTest API change between nightly snapshots.

Step 5.3 - Exit Criterion (met) — PROJECT COMPLETE

- `cmake .. && make -j$(nproc)` builds all targets with zero warnings (`-Wall -Wextra -Wpedantic`). ✓
- `ctest --output-on-failure` passes all 7 GTest executables. ✓
- `./transformer_server -port 8080` starts and logs `Listening on http://0.0.0.0:8080`. ✓
- `POST /predict {"prompt":"1+1="}` returns `{"completion":"2"}` (HTTP 200). ✓
- `GET /health` returns `{"status":"ok"}` (HTTP 200). ✓
- Server handles bad JSON (400), unknown routes (404), wrong method (405), and forward exceptions (500). ✓

20 Bug Fixes & Stabilization (Post-Phase 1)

During the final integration and testing phase, several critical bugs were identified and resolved to achieve a 100% test pass rate.

20.1 DataLoader FileHeader Original Layout Restoration

Issue: `DataLoader: vocab_size mismatch in 'data/math.bin'. Expected 256, got 5085774.`

Root Cause: During earlier stabilization, the C++ `FileHeader` struct was erroneously updated to include a `version` field at offset 4 and shift `vocab_size` to offset 8. However, the existing binary dataset files (`math.bin`, etc.) generated by the Python preprocessor do not possess a `version` field; they strictly use the original 16-byte layout: `magic` (4 bytes), `vocab_size` (4 bytes), and `num_tokens` (8 bytes). Because of the struct mismatch, the data loader read the lower 32-bits of `num_tokens` (5,085,774) and interpreted it as the vocabulary size!

Fix: Reverted the `FileHeader` struct in `dataloader.cpp` back to its original layout (`magic`, `vocab_size`, `num_tokens`) to perfectly align with the binary data format on disk.

20.2 DataLoader Error Handling Use-After-Free (AddressSanitizer)

Issue: Running the executable from the project root under AddressSanitizer triggered a SEGV on unknown address inside `DataLoader::open_and_mmap`.

Root Cause: When a header validation check failed (e.g., magic number or vocabulary size mismatch), the error handling block would immediately call `munmap(mapped_)` to release the memory. However, the subsequent `throw std::runtime_error(...)` statement attempted to format the error message by reading fields from the `hdr` pointer (which pointed into the now-unmapped memory region). This classic use-after-free caused an immediate segmentation fault during the throw.

Fix: Captured the mismatched header values into local `uint32_t` variables *before* invoking `munmap`, ensuring the error message can be safely formatted without dereferencing freed memory.

20.3 FeedForward Phantom GELU Member Compilation Error

Issue: The build failed with error: `'GELU' does not name a type in feedforward.hpp`.

Root Cause: During a previous refactor to resolve `-Wreorder` warnings, a member variable `GELU gelu;` was mistakenly injected into the `FeedForward` class definition. In this C++ engine architecture, GELU is implemented purely as a mathematically fused free function (`engine::nn::gelu`), not as a stateful module class.

Fix: Removed the erroneous `GELU gelu;` declaration from `feedforward.hpp`. The module correctly relies on the free function call during its `forward()` pass.

20.4 CausalSelfAttention Zero-Gradient Mathematical Property

Issue: `test_attention`'s `BackwardGradFlowsToAllParams` assertion failed. It expected the gradient norm of all 8 parameters to be $> 10^{-15}$, but `w_k.bias` had an exactly zero gradient.

Root Cause: In standard scaled dot-product attention, the key bias b_K adds a term $Q \cdot b_K^T$ to the attention scores matrix before the softmax operation. Crucially, this term is constant across all keys for a given query (it depends on row i , not column j). Because the Softmax function is shift-invariant (adding a constant to all elements of a vector does not change the softmax output), b_K has absolutely zero mathematical effect on the forward pass output. Consequently, its true analytical gradient is precisely zero.

Fix: Updated `test_attention.cpp` to explicitly exclude `w_k.bias` from the non-zero gradient check, correctly acknowledging this mathematical property of self-attention.

20.5 FeedForward Constructor Initialization Order

Issue: Compilation emitted `-Wreorder` warnings for the `FeedForward` module.

Root Cause: In C++, member variables are initialized in the exact order they are declared in the class definition, regardless of the order in the constructor initializer list. `d_model_` and `d_ff_` were declared after the `Linear fc1` module, but the constructor

attempted to initialize them first to use their values to construct `fc1`.

Fix: Reordered the member declarations in `feedforward.hpp`, moving `d_model_` and `d_ff_` before the `Linear` and `GELU` sub-modules to match the required initialization order.

21 Interview Discussion: Architectural Trade-offs

21.1 Why "Zero Dependencies"?

Building a Deep Learning engine from the absolute ground up in pure C++17 STL (without external libraries like Eigen, OpenBLAS, or LibTorch) serves as a rigorous proof of fundamental engineering concepts.

Relying purely on the Standard Template Library demonstrates a deep understanding of:

- **Memory & OS Level I/O:** Utilizing POSIX `mmap` for zero-copy file loading and raw `sys/socket.h` for the HTTP server.
- **Data Structures & Lifetimes:** Using `std::shared_ptr` to construct, traverse, and cleanly garbage-collect a Directed Acyclic Graph (DAG) representing the autograd computation without cyclic leaks.
- **Mathematics:** Manually deriving and implementing the backpropagation calculus, Jacobians, and shift-invariant Softmax stability.

21.2 Performance Optimizations: How AVX/BLAS Libraries Accelerate Compute

While the pure STL implementation is architecturally perfect, executing standard $O(N^3)$ nested `for` loops for matrix multiplication (`matmul`) is extremely slow on CPUs. If this engine were optimized for production, replacing the naive C++ loops with a highly optimized Basic Linear Algebra Subprograms (BLAS) library (such as OpenBLAS or Intel MKL) would yield massive performance gains (often $50\times$ – $100\times$ faster).

These libraries achieve performance through three primary mechanisms:

1. **SIMD Vectorization (AVX/AVX-512):** A standard C++ loop executes a single instruction per data point. Modern CPUs contain Advanced Vector Extensions (AVX) registers (up to 512 bits wide). A BLAS library uses raw Assembly to trigger these registers, allowing a single CPU instruction to multiply eight 64-bit `doubles` simultaneously in one clock cycle.
2. **Cache Tiling (Blocking):** Naive matrix multiplication loops cause frequent "cache misses", forcing the CPU to sit idle while waiting for slow Main RAM (≈ 100 ns). BLAS libraries slice matrices into tiny 32×32 sub-blocks (tiles) that fit perfectly inside the ultra-fast L1 CPU Cache (≈ 1 ns). The CPU computes the entire tile instantly without querying Main RAM.
3. **Hardware FMA (Fused Multiply-Add):** Matrix multiplication relies heavily on the operation `sum += A * B`. BLAS leverages the hardware FMA instruction

to perform both the multiplication and the addition in the exact same hardware step, avoiding temporary storage and rounding latency.

21.3 Future Scalability: CPU vs GPU Compute

While BLAS optimizations maximize CPU throughput, scaling this architecture to large parameter counts (e.g., 1B+ parameters) fundamentally requires migrating tensor calculations to a GPU.

- **Architectural Difference:** Modern CPUs feature a small number (e.g., 12–24) of extremely fast, complex cores optimized for low-latency serial branching logic. Conversely, GPUs are designed for high-throughput parallel rendering and feature thousands of simpler cores (e.g., an RTX 3060 contains 3,584 CUDA cores).
- **Matrix Math Parallelism:** Deep Learning operations are overwhelmingly composed of matrix multiplications ($A \times B + C$). Because these arithmetic operations are independent of one another, they can be computed perfectly in parallel. While a CPU processes these operations in small batches across its 12 cores, a GPU can process thousands of output cells simultaneously.
- **Implementation Pathway:** To scale this engine, the core `engine::ops::matmul` function would be rewritten in CUDA C++ or linked against cuBLAS. The autograd DAG logic and structure would remain entirely on the CPU, but the actual tensor buffers would be allocated in GPU VRAM (`cudaMalloc`), data would be transferred over the PCIe bus (`cudaMemcpy`), and calculations would be offloaded to thousands of parallel GPU threads via CUDA kernels.

21.4 1-Hour Curriculum Test Execution Log (1200 Steps)

To definitively prove the engine’s memory safety, dataloader multi-file transitioning, and end-to-end autograd stability, a comprehensive 1200-step test (approximately 1 hour locally) was executed. The engine successfully transitioned across the synthetic math dataset (now supporting subtraction), TinyStories, and Wikipedia without memory leaks. Crucially, the mathematical loss smoothly dropped during all three phases, demonstrating active learning.

C++ Transformer - Curriculum Training

```
d_model      =      64      n_heads   =    4
n_layers     =        2      seq_len  =  128
batch_size   =       32      lr        = 0.001
~params      173696
```

```
Phase 0 MATH   steps [0,      400)
Phase 1 STORIES steps [ 400,   800)
Phase 2 WIKI   steps [ 800,  1200)
```

```
[INFO] Curriculum initialized.
      Total training steps: 1200
```

[INFO] DataLoader opened: data/math.bin
Tokens mapped: 4900039

[INFO] Transformer initialized.
Parameters: 37 tensors

[INFO] AdamW optimizer initialized.
lr=0.001 =0.9 =0.999 =1e-8 =0.01

TRAINING START

step=	0	phase=	MATH	loss=	5.6703	lr=1.00e-03	speed=	0.0 step/s
step=	100	phase=	MATH	loss=	2.4458	lr=1.00e-03	speed=	0.3 step/s
step=	200	phase=	MATH	loss=	2.4025	lr=1.00e-03	speed=	0.3 step/s
step=	300	phase=	MATH	loss=	2.2678	lr=1.00e-03	speed=	0.3 step/s

CURRICULUM TRANSITION: MATH STORIES (step 400)

[CKPT] Saved 1102 KB checkpoint_phase0.bin
[INFO] Dataset swapped to data/stories.bin
Tokens available: 1904212181

step=	400	phase=	STORIES	loss=	11.1010	lr=1.00e-03	speed=	0.3 step/s
step=	500	phase=	STORIES	loss=	2.9067	lr=1.00e-03	speed=	0.3 step/s
step=	600	phase=	STORIES	loss=	2.4937	lr=1.00e-03	speed=	0.3 step/s
step=	700	phase=	STORIES	loss=	2.4287	lr=1.00e-03	speed=	0.3 step/s

CURRICULUM TRANSITION: STORIES WIKI (step 800)

[CKPT] Saved 1102 KB checkpoint_phase1.bin
[INFO] Dataset swapped to data/wiki.bin
Tokens available: 273058991

step=	800	phase=	WIKI	loss=	3.7861	lr=1.00e-03	speed=	0.3 step/s
step=	900	phase=	WIKI	loss=	2.7364	lr=1.00e-03	speed=	0.3 step/s
step=	1000	phase=	WIKI	loss=	2.6644	lr=1.00e-03	speed=	0.3 step/s
step=	1100	phase=	WIKI	loss=	2.6388	lr=1.00e-03	speed=	0.3 step/s

TRAINING COMPLETE (1200 steps)

[CKPT] Saved 1102 KB checkpoint_final.bin

[SUMMARY]

Total steps : 1200
Total time : 4217.3 s
Avg speed : 0.28 step/s
Final ckpt : checkpoint_final.bin

22 Phase 6 - Cloud Deployment & Large-Scale Training

Phase 6 Goal

Deploy the C++ Deep Learning Engine to a high-performance Google Cloud Platform (GCP) Compute Engine instance to perform a 7-day continuous training run on a 3.1 million parameter model across a 50,000-step curriculum.

22.1 Cloud Hardware Selection

Due to local hardware constraints (8-core laptop without sustained thermal headroom), training was migrated to a Google Cloud `c2-standard-8` instance. The C2 instance family features Intel Cascade Lake processors with high memory bandwidth and AVX-512 vector instructions, ensuring maximal throughput for the C++ engine's matrix operations without thermal throttling during the 168-hour run.

22.2 Deployment Pipeline

The deployment process utilised the Google Cloud CLI (`gcloud`) directly from the local Windows Subsystem for Linux (WSL) environment:

1. **Provisioning:** A 50 GB Ubuntu 22.04 LTS boot disk was selected to accommodate the 4.3 GB `.bin` dataset.
2. **Secure Transfer:** The entire `Project` directory was transferred using `gcloud compute scp -recurse`.
3. **Compilation:** To fully leverage the cloud architecture, the engine was recompiled directly on the VM using `cmake` and `make`.

22.3 The "Golden Ratio" Training Command

To fit a meaningful curriculum into exactly 7 days (168 hours) on 8 CPU cores, the model dimensions and step thresholds were mathematically balanced. A 3.1 million parameter architecture was selected:

- `d_model` = 256
- `n_heads` = 4
- `n_layers` = 4

The curriculum was scheduled for 50,000 total steps:

- **Math Phase:** 5,000 steps
- **Stories Phase:** 20,000 steps (Transition at 5,000)
- **Wikipedia Phase:** 25,000 steps (Transition at 25,000; Total 50,000)

The training was launched asynchronously using `nohup` to ensure persistence across SSH session disconnects:

```
nohup ./build/transformer --d_model 256 --n_heads 4 --n_layers 4 \
--phase_steps 5000,25000,50000 > training.log 2>&1 &
```

Listing 24: Cloud Training Command

Project Complete — Full Repository Summary

Component	Key files	Lines
Math Engine	tensor, node, ops, autograd	≈ 1,800
Neural Modules	linear, layernorm, attention, transformer	≈ 2,200
Data Pipeline	dataloader, curriculum	≈ 600
Optimizer	AdamW	≈ 200
Loss	Fused cross-entropy	≈ 150
Training Loop	main.cpp	≈ 300
Tests (GTest)	7 files, 52+ assertions	≈ 1,000
Benchmarks	matmul, attention, dataloader	≈ 450
Server	http_server, inference_handler	≈ 500
Build	CMakeLists.txt	≈ 130
Total		≈ 7,300

Table 23: Complete repository inventory — C++17, zero external runtime dependencies.